Kingdom of Saudi Arabia
Ministry of Education
Imam Abdulrahman bin Faisal University
College of Science and Humanities-Jubail
Computer science Department

**Software Engineering**
**Term 1 – 2024/2025**

# Software Design Specifications

## For

# IAU WiseOwl

**Version [1.0]**

**Students Name:**
Jumanh Alhashem
Hawra ALmahfouz
Aseel Alqahtani

**Advisor:** Dr.Taghreed AL-Tarmoom

**24/11/2024**

This Software Design Specification (SDS) has been prepared as an official deliverable for the Software Engineering course (CS 411), Term 1. It will serve as a core reference document for the IAU WiseOwl platform throughout the entire development lifecycle. This document is prepared in alignment with the IEEE Recommended Practice for Software Design Descriptions, ensuring consistency with industry standards for structured system design and engineering.

**Members of the group:**


| # | Name | ID | Role |
|---|---|---|---|
| **1** | Jumanh Alhashem | 2240001824 | Leader |
| **2** | Hawra ALmahfouz | 2240002209 | Member |
| **3** | Aseel Alqahtani | 2240005478 | Member |


## Revision History

| Name | Date | Reason For Changes | Version |
|---|---|---|---|
| **All members** | Nov 10, 2025 | Prepared initial version of the SDS document | 0.1 |
| **All members** | Nov 12, 2025 | Updated Section 3: Design Considerations | 0.2 |
| **All members** | Nov 14, 2025 | Added System Architecture & Data Design | 0.3 |
| **All members** | Nov 16, 2025 | Updated UI Design Screens and Interfaces | 0.4 |
| **All members** | Nov 18, 2025 | Completed Detailed Subsystem Design | 0.5 |
| **All members** | Nov 24, 2025 | Final review – Complete and approved version | 1.0 |

## Table of Contents

1. Introduction.....7
1.1 Purpose.....7
1.2 Scope.....7
1.3 Definitions, Acronyms, and Abbreviations.....8
1.4 References.....9

2. System Overview.....9
2.1 WiseOwl System Context.....9
2.2 Major Features and Functional Modules.....12
2.3 User Classes and Typical Scenarios.....13

3. Design Considerations.....14
3.1 Assumptions and Dependencies.....14
3.1.1 Related Hardware and Software.....14
3.1.2 Operating Systems and Supported Platforms.....15
3.1.3 End-User Characteristics.....15
3.1.4 Expected Functional Changes (Prototype Evolution).....16
3.2 General Constraints.....16
3.2.1 Hardware / Software Constraints.....16
3.2.2 Interface and Integration Constraints.....17
3.2.3 Data Storage and Distribution Constraints.....17
3.2.4 Security and Protection Constraints.....17
3.2.5 Performance Requirements.....17
3.2.6 Verification and Validation Requirements.....17

4. User Interface & Prototype Design.....18
4.1 Prototype Overview.....18
4.2 UI Design Standards and Guidelines.....19
4.3 Screen Images (Prototype Screens).....19

5. System Architecture.....26
5.1 Architectural Approach.....26
5.2 Overall Architectural Design.....27
5.3 Subsystem Architecture.....28
5.3.1 Presentation Layer.....29
5.3.2 Business Logic Layer.....29
5.3.3 Data Access & Persistence Layer.....30
5.3.4 External Integration Layer.....30

6. Data Design.....32

6.1 Data Description.....32
6.2 Data Dictionary.....33
6.3 Database Schema.....34

7. Component Design.....36
7.1 Common Components.....36
7.1.1 Authentication & Session Management.....37
7.1.2 Authorization & Roles.....37
7.1.3 Notification & Messaging.....38
7.2 Core WiseOwl Components.....38
7.2.1 Student Management.....39
7.2.2 Schedule Management.....39
7.2.3 Reminders Management.....40
7.2.4 Internship Management.....40
7.2.5 Announcements Management.....40
7.2.6 Communication / Messaging.....41
7.3 Supporting Components.....41
7.3.1 Settings & Configuration.....41
7.3.2 Backup & Synchronization.....42
7.3.3 Logging & Monitoring.....42

8. Detailed System Design.....43
8.1 Classification.....43
8.2 Definitions.....43
8.3 User Interactions & Use Flows.....44
8.4 Resource Requirements.....44
8.5 Processing Logic.....45
8.6 Interfaces / Exports.....45
8.7 Subsystem Detailed Design.....46
8.7.1 Authentication & User Management Subsystem.....46
8.7.2 Schedule & Reminders Subsystem.....47
8.7.3 Internship Tracking Subsystem.....48
8.7.4 Announcements Subsystem.....49
8.7.5 Messaging Subsystem.....49
8.7.6 Profile & Settings Subsystem.....50
8.8 Responsibilities.....50
8.9 Constraints.....50
8.10 Composition.....51

9. Additional Design Features.....51
9.1 Performance & Scalability.....52
9.2 Security Architecture.....52

9.3 Error Handling & Fault Recovery.....53
9.4 Localization (Arabic / English).....54
9.5 Maintainability & Extensibility.....55

10. Requirements Traceability Matrix.....55
10.1 Traceability Matrix.....56

## Table of Tables

Table 1: WiseOwl Terminologies and Definitions.....8
Table 2: Acronyms and Abbreviations.....9
Table 3: User Classes and Characteristics.....14
Table 5: Data Description Overview.....32
Table 6: Data Dictionary.....33
Table 7: Component Responsibilities.....42
Table 8: Component Constraints.....42
Table 9: System Resource Requirements.....44
Table 10: Prototype Usability Metrics.....52
Table 13: Roles and Permissions Matrix.....38
Table 14: Supported Devices and Platforms.....15
Table 15: Data Types for Schedule & Reminders.....39
Table 16: User Settings and Configuration Parameters.....41
Table 17: Internship Status Categories.....40
Table 18: Announcement Categories.....41
Table 19: API Integrations.....31
Table 20: System Availability SLA.....52
Table 21: Backup and Recovery Protocol.....54
Table 22: Localization Specifications.....54
Table 23: Prototype Evaluation & Testing Checklist.....18

## Table of Figures

Figure 1: WiseOwl System Context Diagram.....10
Figure 2: Context-Level DFD.....11

Figure 3: Level-0 DFD.....12
Figure 4: Home / Landing Screen.....20
Figure 5: Login Screen.....20
Figure 6: Student Registration Screen.....21
Figure 7: Student Home Dashboard.....21
Figure 8: Reminders List Screen.....22
Figure 9: Add New Reminder Interface.....22
Figure 10: Internship Status Tracking Screen.....23
Figure 11: Class Schedule.....23
Figure 12: University Announcements List Screen.....24
Figure 13: Profile Screen.....24
Figure 14: Settings Screen.....25
Figure 15: Error & System Feedback Screen Samples.....26
Figure 16: Layered Architecture Diagram.....28
Figure 17: ER Diagram.....34
Figure 18: Relational Schema.....35
Figure 19: High-Level Component Diagram.....36
Figure 20: Sequence Diagram – Login.....46
Figure 21: Sequence Diagram – Schedule Management.....47
Figure 22: Sequence Diagram – Internship Process.....48
Figure 23: Sequence Diagram – Messaging & Notifications.....49

# 1. Introduction

IAU WiseOwl is an academic companion platform designed to help university students organize and manage their daily academic life in one place. The system provides a personalized dashboard that shows today's classes, upcoming events, and important updates. Students can view and manage their weekly class schedule, set reminders for exams, assignments, and registration deadlines, and receive timely notifications.

WiseOwl also connects students with internship and cooperative training opportunities by allowing them to browse, filter, and apply for available positions. In addition, the platform delivers official announcements from departments and colleges and supports secure in-app messaging between students and academic staff such as advisors and coordinators. By bringing schedules, reminders, internships, announcements, and communication together into a single, easy-to-use interface, WiseOwl aims to reduce information fragmentation and improve students' academic planning and engagement.

## 1.1 Purpose

The purpose of this Software Design Specification (SDS) is to define, in a structured and detailed manner, the design of the **IAU WiseOwl** system.
This document translates the approved functional and non-functional requirements into a technical blueprint that will guide implementation, integration, testing, deployment, and maintenance activities.

The SDS is intended for:

- Software developers and architects, who will use it as the primary reference during design and implementation.
- Test engineers, who will derive test cases and scenarios to verify that the implementation conforms to the design.
- Project managers and supervisors, who require a clear understanding of the system structure, modules, and interfaces to plan and monitor progress.
- Future maintainers and stakeholders, who need an authoritative description of how the system is organized and how components interact.

By providing a consistent view of the system architecture, data model, components, and interactions, this document aims to reduce ambiguity, improve communication among stakeholders, and ensure that the final implementation of IAU WiseOwl aligns with its intended objectives.

## 1.2 Scope

This Software Design Specification describes the design of **IAU WiseOwl**, a web-based and mobile-friendly academic companion platform designed primarily for university students. The system consolidates key academic activities into a single, coherent environment by offering the following core capabilities:

- A personalized **dashboard** that summarizes today's classes, upcoming events, active reminders, recent announcements, and internship updates.
- **Class schedule management**, providing a clear weekly view of courses with times, locations, and instructors.
- **Reminder and notification management**, allowing students to configure and receive notifications for exams, assignments, registration deadlines, and other academic tasks.
- **Internship and cooperative training management**, enabling staff to publish opportunities and students to browse, filter, and apply for suitable positions.
- **Announcement management**, where departments and colleges can broadcast official messages and students can view and search announcements relevant to them.
- **In-app messaging**, supporting secure, structured communication between students and authorized academic staff such as advisors, coordinators, and internship supervisors.

- **Profile and settings management**, through which users can maintain their personal information, academic details, notification preferences, and localization options.

The scope of this document includes the logical and physical architecture of these modules, the data structures and database schema that support them, and the software components and interfaces required for their operation. It details how the system's layers (presentation, application, service, and data) collaborate to deliver the above functionality.

The following aspects are **outside** the scope of IAU WiseOwl as defined in this SDS:

- Management of course grades, transcripts, or full learning-management functionality (e.g., assignment submission, content delivery).
- Financial processes such as tuition payments or stipend disbursement.
- Generic social-network features unrelated to academic support.

Any extensions beyond the functions described above are considered future enhancements and are not covered by this version of the SDS.

## 1.3 Definitions, Acronyms, and Abbreviations

This section provides standardized definitions for terminology used throughout this SDS to avoid ambiguity and ensure consistent interpretation across all stakeholders.

**WiseOwl Terminologies and Definitions**

| Term | Definition |
|---|---|
| **WiseOwl** | The academic companion platform described in this SDS. It centralizes class schedules, reminders, internships, announcements, messaging, and user settings for university students. |
| **Student** | A primary end user of WiseOwl who logs in to view schedules, configure reminders, apply for internships, read announcements, and communicate with academic staff. |
| **Academic Staff / Coordinator** | A user with elevated privileges (e.g., advisor, course instructor, internship coordinator) who can publish announcements and internships and communicate with students through the system. |
| **Administrator** | A user responsible for managing accounts, roles, and global configuration settings of WiseOwl and ensuring system security and data integrity. |
| **Dashboard** | The main landing screen that provides a summarized view of today's classes, upcoming events, reminders, announcements, and internship updates for the logged-in student. |
| **Class Schedule** | The structured weekly timetable of a student's courses, including course codes, names, times, locations, and instructors. |
| **Reminder** | A user-defined alert associated with an academic task (e.g., exam, assignment, registration deadline) that triggers a notification at a specified time. |
| **Notification** | A message generated by the system to inform users about upcoming reminders, new announcements, internship status changes, or unread messages, delivered in-app and/or via external channels when configured. |

| | |
|---|---|
| **Internship Opportunity** | A cooperative training or internship position published through WiseOwl by academic staff or coordinators, which students can browse and apply for. |
| **Internship Application** | A record representing a student's application to a specific internship opportunity, including status information such as pending, accepted, or rejected. |
| **Announcement** | An official message authored by departments, colleges, or program coordinators and broadcast to selected groups of students through WiseOwl. |
| **Message / Conversation Thread** | A structured exchange of messages between a student and one or more academic staff members within the WiseOwl messaging module. |
| **User Profile** | The set of personal, academic, and preference data associated with a WiseOwl account, maintained by the user and/or administrators. |
| **Settings** | Configuration options that allow users to customize language, notification preferences, and other aspects of their experience in WiseOwl. |
| **User Role** | A predefined set of permissions (e.g., Student, Academic Staff, Administrator) that determines the actions a user can perform in the system. |

## 1.4 References

The following documents, standards, and resources were referenced during the preparation of this SDS:

1. **WiseOwl Software Requirements Specification (SRS), Version 1.0** – Primary source of functional and non-functional requirements.
2. **IEEE 1016-2009** – IEEE Standard for Software Design Descriptions.
3. **IEEE 830-1998** – IEEE Recommended Practice for Software Requirements Specifications.
4. **UML 2.5 Specification** – Unified Modeling Language guidelines for diagrams used in this document.
5. **RESTful API Architectural Constraints** – Design principles for service communication.
6. **Database Design Best Practices (IBM, Oracle, PostgreSQL Documentation)** – Referenced for schema normalization and relational modeling.

# 2. System Overview

The WiseOwl platform is designed as an integrated academic companion that centralizes key student activities such as schedule viewing, reminder management, internship applications, announcement tracking, and secure communication with academic staff. This section provides a system-level perspective that explains how WiseOwl fits within its operational environment, outlines its major functional modules, and describes the main user classes that interact with the system. The goal is to establish a clear understanding of the platform as a cohesive system before presenting detailed architectural, data, and component-level designs in later sections.

## 2.1 WiseOwl System Context

WiseOwl operates as a web-based platform accessible through modern browsers on desktop and mobile devices. It acts as a centralized layer between university information sources (such as class schedules, internship postings, and announcements) and students who consume and manage this information.

WiseOwl interacts with the following primary external entities:

1. **Students** – Access the platform to view their weekly schedule, configure reminders, browse and apply for internships, read announcements, and exchange messages with academic staff.
2. **Academic Staff / Coordinators** – Publish announcements and internship opportunities, review internship applications, and communicate with students through the messaging module.
3. **System Administrators** – Manage user accounts, roles, and global configuration settings, and oversee the overall health and data integrity of the system.
4. **University Authentication Service** – Provides secure authentication (e.g., SSO or centralized university login) and returns verified identity information for users attempting to access WiseOwl.
5. **Notification Services (Optional)** – External services (such as email or push notification gateways) that deliver notifications to users outside the application when configured.

Conceptually, WiseOwl sits at the center of these entities, receiving data such as schedules, announcements, and internship information from authorized staff and administrators, processing it through its internal modules, and presenting personalized, student-centric views and notifications to end users.

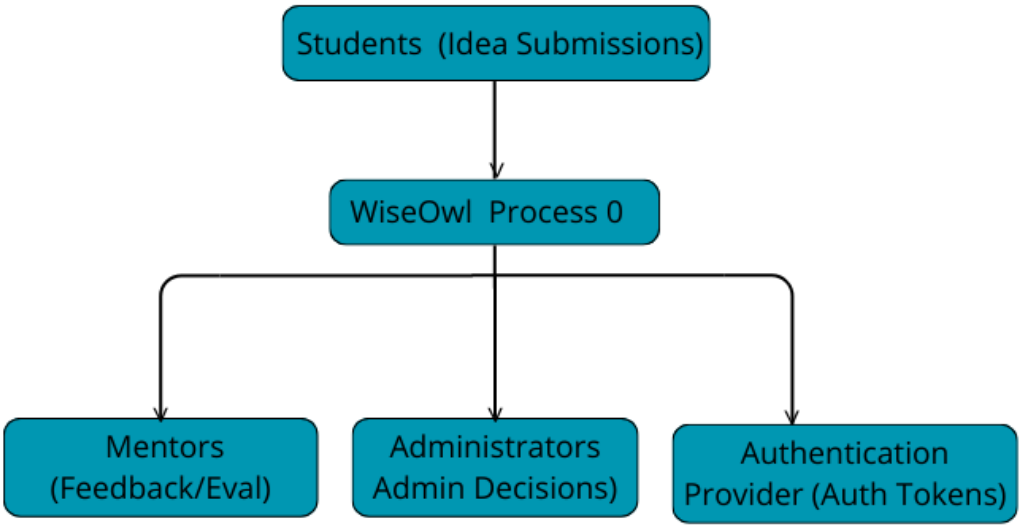



Figure 1: WiseOwl System Context Diagram

**Context-Level Data Flow Diagram (DFD)**

The context-level DFD visualizes WiseOwl as a single process interacting with external entities. It identifies the key data exchanges but does not show internal processes or data stores.

Key data exchanges include:
– Idea submissions from students
– Feedback and evaluation from mentors
– Administrative decisions from administrators
– Authentication tokens from the authentication provider

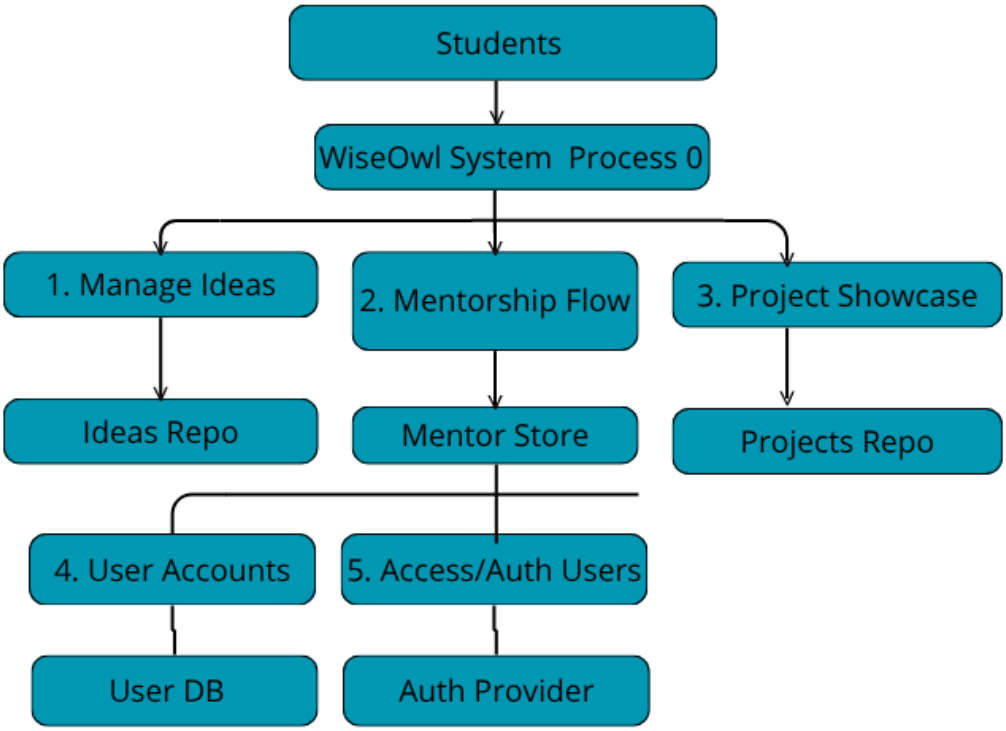



Figure 2: Context-Level DFD

**Level-0 Data Flow Diagram (DFD)**

The Level-0 DFD decomposes the system into major functional processes, data stores, and external interactions. The typical Level-0 processes include:

1. **Manage Ideas**
2. **Manage Mentorship Workflow**
3. **Manage Project Showcase**
4. **Manage User Accounts and Roles**
5. **Access/Authenticate Users**

Data stores that appear in the Level-0 diagram include:
– User Database
– Ideas Repository
– Mentor Feedback Store
– Projects Repository

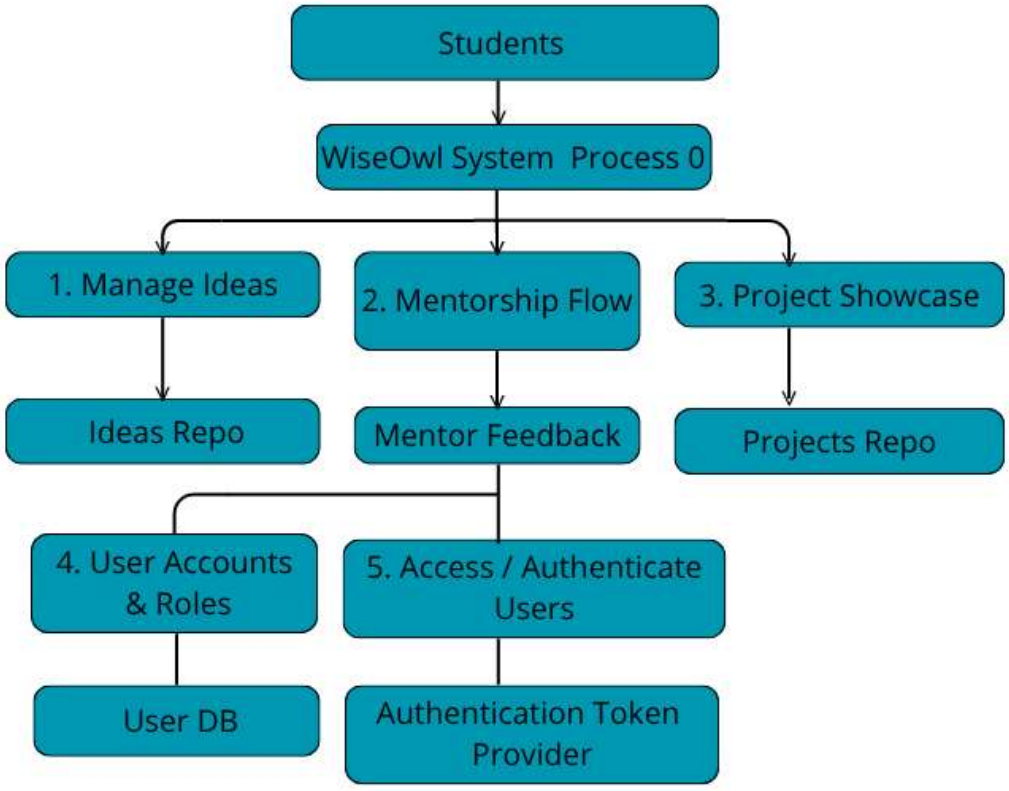



Figure 3: Level-0 DFD

## 2.2 Major Features and Functional Modules

The WiseOwl platform is decomposed into several functional modules, each responsible for a distinct aspect of the system's behavior. These modules collaborate through well-defined interfaces to deliver a seamless user experience:

1. **Authentication & Role Management Module**
   - Handles secure login and logout.
   - Integrates with the university authentication service.
   - Assigns and enforces roles (Student, Academic Staff, Administrator).
2. **Dashboard Module**
   - Provides a personalized overview of today's classes and upcoming events.
   - Aggregates reminders, recent announcements, and internship updates in a single view.
3. **Schedule Management Module**
   - Stores and displays students' weekly class schedules.
   - Allows students to browse their schedule by day and week.
   - Supports viewing course details such as instructor, time, and location.
4. **Reminders & Notifications Module**
   - Allows students to create, update, and delete reminders for exams, assignments, registration deadlines, and other academic tasks.

- Triggers notifications according to user preferences and configured time offsets.

5. **Internship & Training Management Module**
   - Enables academic staff or coordinators to publish internship and cooperative training opportunities.
   - Allows students to browse, filter, and apply for opportunities.
   - Tracks application status and important dates (submission deadlines, interviews, acceptance).
6. **Announcement Management Module**
   - Allows departments and colleges to create and target announcements to specific groups of students.
   - Provides students with a searchable and filterable feed of announcements.
7. **Messaging Module**
   - Supports secure, structured messaging between students and authorized academic staff.
   - Maintains conversation history and basic metadata (timestamps, sender, read status).
8. **Profile & Settings Module**
   - Stores and manages user profile information and academic details.
   - Provides configuration options such as language, notification preferences, and privacy settings.

Together, these modules implement the core functional behavior of WiseOwl and form the basis for the detailed architectural and component-level design described in subsequent sections.

### 2.3 User Classes and Typical Scenarios

WiseOwl primarily serves three user classes, each with distinct goals and interaction patterns:

1. **Students**
   - View their class schedule and daily agenda.
   - Configure reminders for exams, assignments, and other academic events.
   - Browse and apply for internships and cooperative training opportunities.
   - Read official announcements relevant to their college, department, or program.
   - Communicate with academic staff through the messaging module.
2. **Academic Staff / Coordinators**
   - Publish and manage internship opportunities and related information.
   - Create and broadcast announcements to targeted groups of students.
   - Review internship applications and update their status.
   - Respond to student queries and provide guidance through in-app messaging.
3. **System Administrators**
   - Manage user accounts, roles, and access permissions.
   - Configure system-wide settings and integrations (e.g., authentication, notification services).
   - Monitor system usage, logs, and performance to ensure reliability and security.

These user classes and scenarios guide the design decisions presented in the rest of this SDS and ensure that WiseOwl remains aligned with its primary goal: supporting students in organizing and managing their academic activities effectively.

**User Classes and Characteristics**

| User Class | Characteristics | Permissions | Typical Scenarios |
|---|---|---|---|
| **Student** | Typically undergraduate/graduate students; content creators. | Submit ideas, upload documents, communicate with mentors, publish projects. | Submitting a new idea; receiving mentor feedback; updating project details. |
| **Mentor** | Subject-matter experts assigned to guide students. | View submissions, add comments, evaluate proposals, track student progress. | Reviewing a student's idea; submitting feedback; approving a final proposal. |
| **Administrator** | System supervisors ensuring platform stability. | Manage users, oversee submissions, configure system settings, generate reports. | Activating new mentor accounts; approving project publication; reviewing flagged content. |
| **Guest / Visitor** | External viewers with restricted access. | View public project showcases only. | Browsing completed student projects. |

# 3. Design Considerations

This section outlines the foundational design considerations that influence the architectural, functional, and operational design decisions for the WiseOwl platform. These considerations ensure that the system is feasible, maintainable, scalable, secure, and aligned with the intended user environment. They address assumptions, dependencies, constraints, performance expectations, and requirements for verification and validation.

## 3.1 Assumptions and Dependencies

The design of the WiseOwl platform is based on specific assumptions and system dependencies that must hold true for successful implementation and operation. These elements define the technological expectations and environmental conditions under which the system will function.

### 3.1.1 Related Hardware and Software

The WiseOwl system relies on a combination of hardware devices and software services for optimal performance. These include:

**Related Hardware**
– Personal computers (laptops/desktops) with modern browsers
– Smartphones and tablets supporting mobile web usage

– Backend servers for hosting APIs, authentication services, and data storage
– University network infrastructure supporting stable internet availability

**Related Software**
– Browser support (Chrome, Firefox, Safari, Edge)
– A DBMS such as PostgreSQL or MySQL
– Backend runtime environment (Node.js / Python)
– Authentication middleware (OAuth 2.0 / University SSO)
– Cloud hosting platform (optional): AWS, Azure, or GCP
– API clients supporting HTTPS and JSON data exchange

These hardware and software relationships directly influence the architecture, UI design, performance decisions, and security mechanisms.

### 3.1.2 Operating Systems and Supported Platforms

WiseOwl is designed as a cross-platform, web-based system accessible from major operating systems and devices. The platform emphasizes responsive web design, ensuring seamless functionality across device types and display sizes.

**Supported Devices and Platforms**

| Device Category | Examples | Supported OS | Notes |
|---|---|---|---|
| **Desktop/Laptop** | PCs, MacBooks | Windows, macOS, Linux | Full access to all features |
| **Tablets** | iPad, Samsung Tab | iPadOS, Android | Optimized interface for medium screens |
| **Smartphones** | iPhone, Android phones | iOS, Android | Responsive UI; supports full user workflows |
| **Servers** | Cloud or on-premise | Linux (preferred), Windows Server | Hosts backend services and DBMS |

### 3.1.3 End-User Characteristics

Understanding end-user characteristics helps refine UI/UX, performance design, and accessibility strategies.

**Students:**
– Moderate technical proficiency
– Frequent use of smartphones
– Require intuitive workflows for idea submission

**Mentors:**
– Higher domain expertise
– Require structured access to student ideas
– Prefer simplified feedback interfaces

**Administrators:**
– Skilled in academic and administrative workflows
– Require advanced dashboards for monitoring and decision-making

**Accessibility Considerations:**
– Clear visual hierarchy
– Compatibility with assistive technologies
– Multi-language support (optional)

These characteristics ensure the design aligns with user expectations and minimizes cognitive load.

### 3.1.4 Expected Functional Changes (Prototype Evolution)

The WiseOwl platform is expected to evolve across multiple iterations of the prototype. Anticipated changes include:

1. **Enhanced Mentor–Student Interaction Tools**
   – Expanded commenting features
   – Real-time chat or messaging (future release)
2. **Advanced Analytics**
   – Student progress dashboards
   – Mentor performance metrics
3. **AI-Driven Feedback (Long-term Evolution)**
   – Automated idea screening
   – Suggested improvements for submissions
4. **Integration with Learning Management Systems (LMS)**
   – Synchronization with Moodle, Blackboard, or Canvas
5. **Mobile App Development (Optional Future Phase)**
   – Native iOS/Android applications

Such evolutionary enhancements influence the need for modular services, scalable database design, and loosely coupled architectural components.

## 3.2 General Constraints

The WiseOwl platform must operate within several constraints affecting design, implementation, and system performance. These include hardware limitations, integration boundaries, data policies, security controls, and testing requirements.

### 3.2.1 Hardware / Software Constraints

– Limited computational capabilities on older user devices require optimized frontend rendering.
– Backend services must operate within the resource limits of the hosting environment.
– Database storage must adhere to institutional policies on record retention and archival timelines.
– Browser compatibility requires supporting HTML5, CSS3, and ES6 JavaScript standards.

These constraints drive the need for lightweight UI components, efficient backend API design, and optimized queries.

### 3.2.2 Interface and Integration Constraints

– Integration with external authentication providers (OAuth 2.0 or university SSO) mandates adherence to established protocols.
– API endpoints must follow RESTful architectural guidelines.
– JSON is required as the standard communication format between frontend and backend.
– External LMS or third-party tools may impose rate limits or access rules.

These constraints guide the integration architecture and limit alternative design options.

### 3.2.3 Data Storage and Distribution Constraints

– Sensitive student and mentor data must be stored in encrypted form.
– Data must be partitioned logically to separate ideas, feedback, and user records.
– Backups must occur according to institutional policy (daily or weekly).
– Distributed storage is optional but recommended for scalability.

Such constraints influence database schema design, indexing strategies, and backup planning.

### 3.2.4 Security and Protection Constraints

Security is a non-negotiable design constraint. The system must comply with the following:

– Passwords must never be stored in plain text; hashing with modern algorithms (e.g., bcrypt).
– All communication must use HTTPS/TLS encryption.
– Role-based Access Control (RBAC) must be enforced on every module.
– Audit logs must be maintained for all mentor and administrator actions.
– Protection against SQL injection, XSS, CSRF, and brute-force attacks.

These constraints govern both backend and frontend security design.

### 3.2.5 Performance Requirements

The WiseOwl platform is expected to meet the following performance benchmarks:

– Page load time: less than 2.5 seconds on standard broadband.
– API response time: under 500 milliseconds for CRUD operations.
– System availability: 99.5% uptime.
– Maximum concurrent users: scalable to 500+ without performance degradation.
– Database queries must be optimized to operate under 100 ms for typical read/write workloads.

Meeting these performance constraints ensures a high-quality user experience.

### 3.2.6 Verification and Validation Requirements

Verification and validation (V&V) ensure that the system meets the design specifications and satisfies user needs.

Verification focuses on reviewing architecture, APIs, databases, and UI components.
Validation ensures the system functions according to user expectations through prototype testing.

**Prototype Evaluation & Testing Checklist**

| Category | Verification Activities | Validation Activities | Expected Outcome |
|---|---|---|---|
| **UI/UX** | Screen reviews, workflow inspection | Usability tests with students/mentors | Intuitive and error-free navigation |
| **API Layer** | Endpoint inspection, unit tests | Integration tests | Correct data exchange and processing |
| **Database** | Schema review, constraint validation | Real-world data testing | Stable, consistent storage operations |
| **Security** | Code scanning, vulnerability tests | Penetration testing | Robust protection against threats |
| **Performance** | Load testing, query optimization | Stress testing | Stable system under peak loads |
| **Functional Requirements** | Requirement-to-design traceability | End-user task validation | System performs expected functions |

## 4. User Interface & Prototype Design

This chapter presents the full user interface and prototype design for the WiseOwl platform. It explains the design philosophy, visual standards, and detailed descriptions of all prototype screens. These screens serve as the foundational reference for developers and designers implementing the interface.

### 4.1 Prototype Overview

The WiseOwl prototype visually represents the UI layout, navigation structure, and functionality of the platform. It is designed to support intuitive, accessible, and efficient interactions for students, mentors, and administrators.

**Primary Objectives**

- Deliver a **user-friendly, modern, and responsive** interface.
- Provide **clear navigation and organized screen flows**.
- Support **multilingual usage**, including English and Arabic.
- Maintain **visual consistency** across modules using unified design rules.
- Represent **task-oriented workflows** aligned with academic operations.

The prototype demonstrates the platform's major functional modules: authentication, dashboard, class scheduling, reminders, announcements, internships, messaging, profile management, settings, and onboarding.

## 4.2 UI Design Standards and Guidelines

The WiseOwl UI is guided by established design standards to ensure clarity, ease of use, and accessibility.

### Color Palette

- **Primary:** Blue tones for an academic and trustworthy look.
- **Secondary:** Light gray and white backgrounds for clean layouts.
- **Accents:** Green, yellow, or orange to highlight reminders and alerts.

### Typography

- **Headings:** Large, bold, highly readable fonts.
- **Body Text:** Medium weight, 14–16px.
- **Alignment:** Clean left-aligned (LTR) for English; mirrored RTL for Arabic.

### Layout Rules

- Card-based modular content blocks.
- Uniform spacing, padding, and margins.
- Clear separation between navigation and content zones.

### Iconography

- Vector icons for clarity and scalability.
- Universal icons for common actions (schedule, alerts, messages, etc.).

### Accessibility

- Support for both **English and Arabic**.
- High contrast ratios.
- Accessible forms with labels and error messages.

### Interaction Behavior

- Hover and active states for interactive elements.
- Smooth page transitions.
- Clear confirmation dialogs and error messages.

## 4.3 Screen Images (Prototype Screens)

## 4.3.1 Landing & Authentication Interfaces

### 4.3.1.1 Home / Landing Screen

The entry point for users featuring key highlights, login, and registration links.




Figure 4: Home / Landing Screen

### 4.3.1.2 Login Screen

Allows users to log in using email and password or SSO (if integrated).

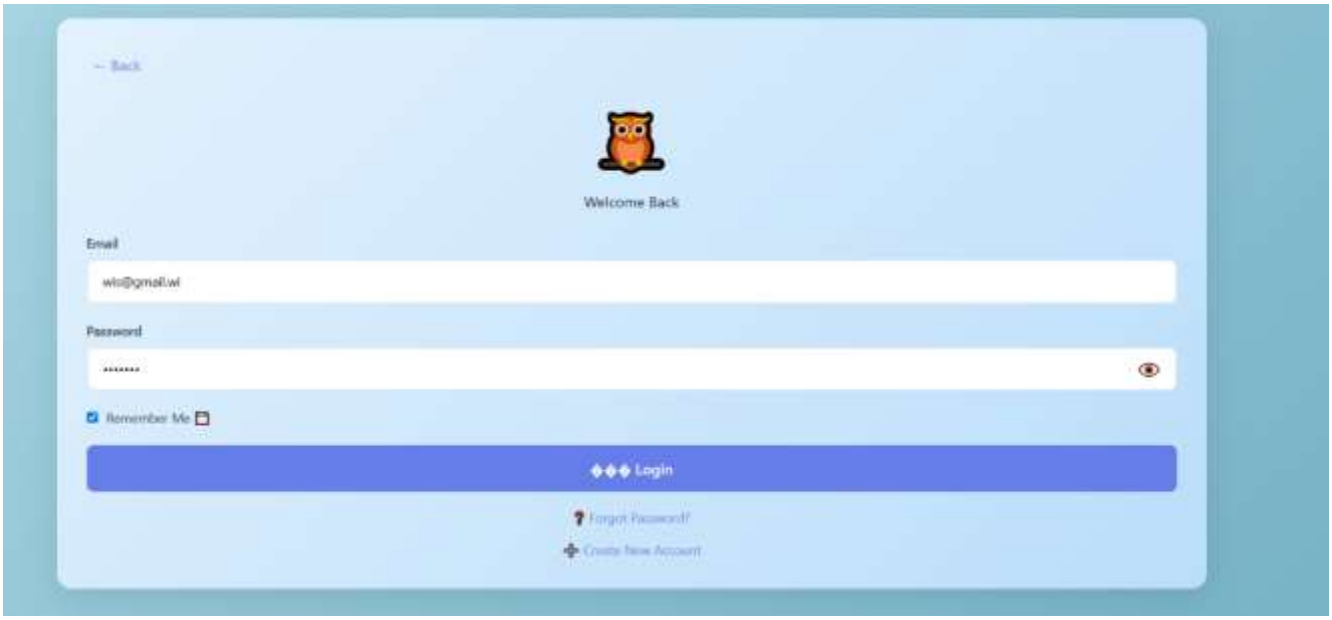



Figure 5: Login Screen

#### 4.3.1.3 Student Registration Screen

Captures student data including name, email, academic details, and password.

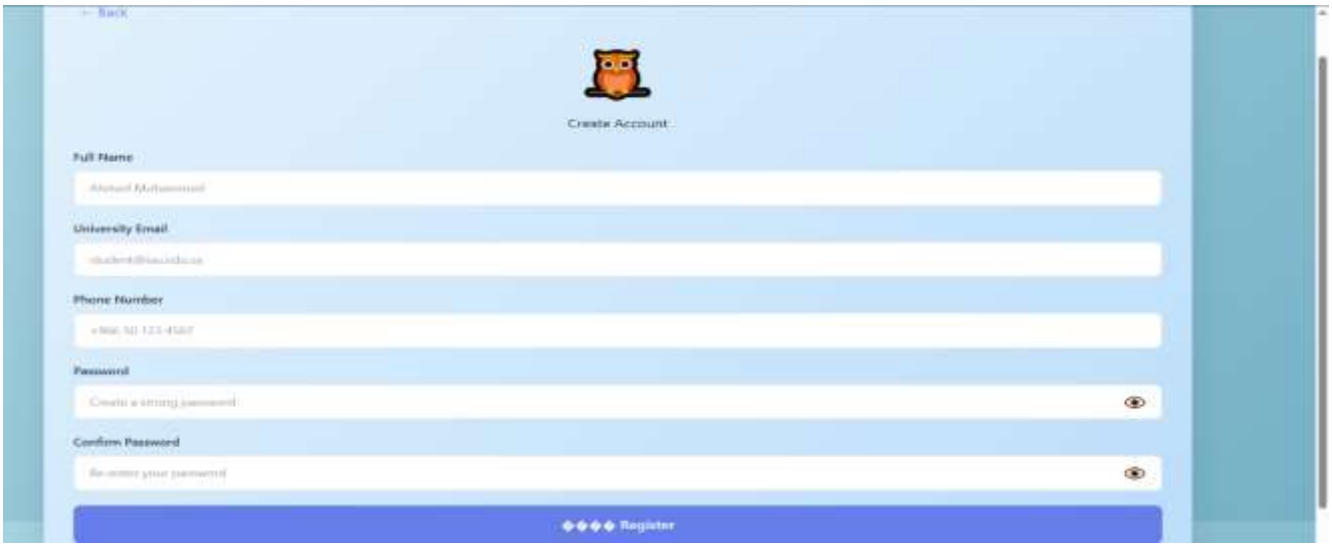



Figure 6: Student Registration Screen

### 4.3.2 Dashboard & Navigation

#### 4.3.2.1 Student Home Dashboard

Displays classes, reminders, announcements, and quick navigation.

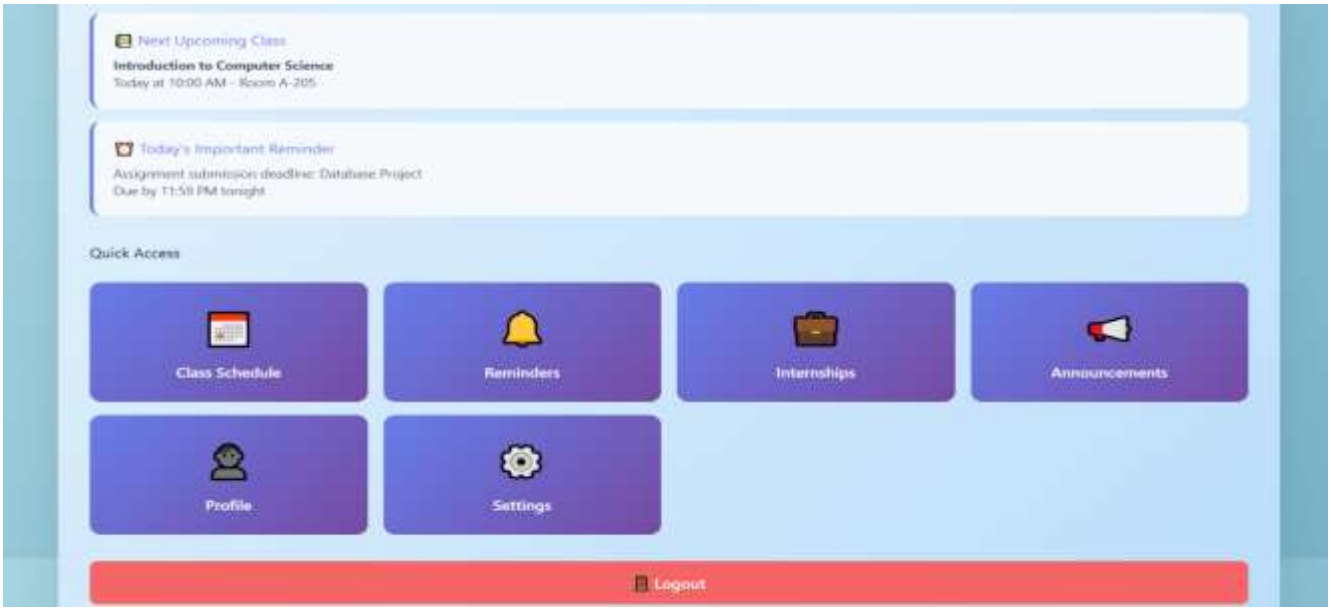



Figure 7: Student Home Dashboard

## 4.3.4 Reminders & Notifications Interfaces

### 4.3.4.1 Reminders List Screen

Shows upcoming tasks and academic deadlines.

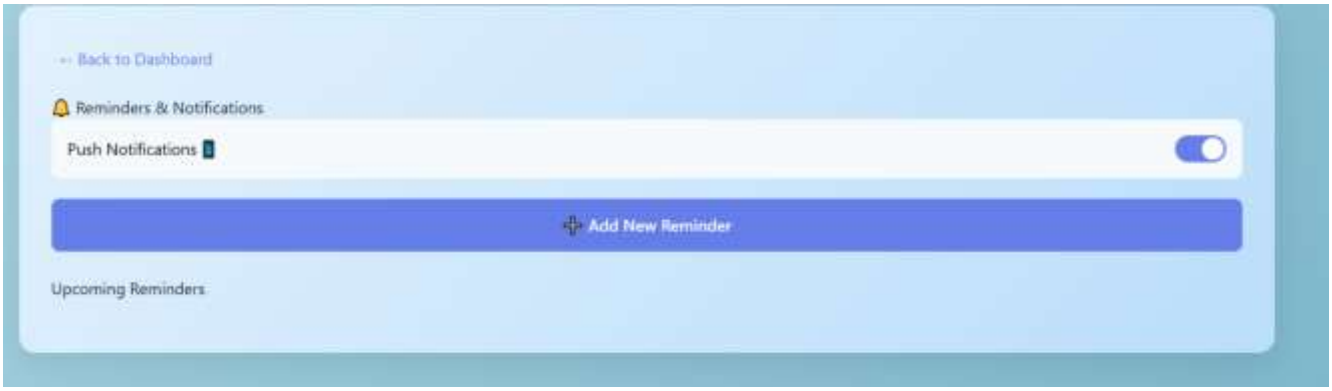



Figure 8: Reminders List Screen

### 4.3.4.2 Add New Reminder Interface

Form for creating new reminders.

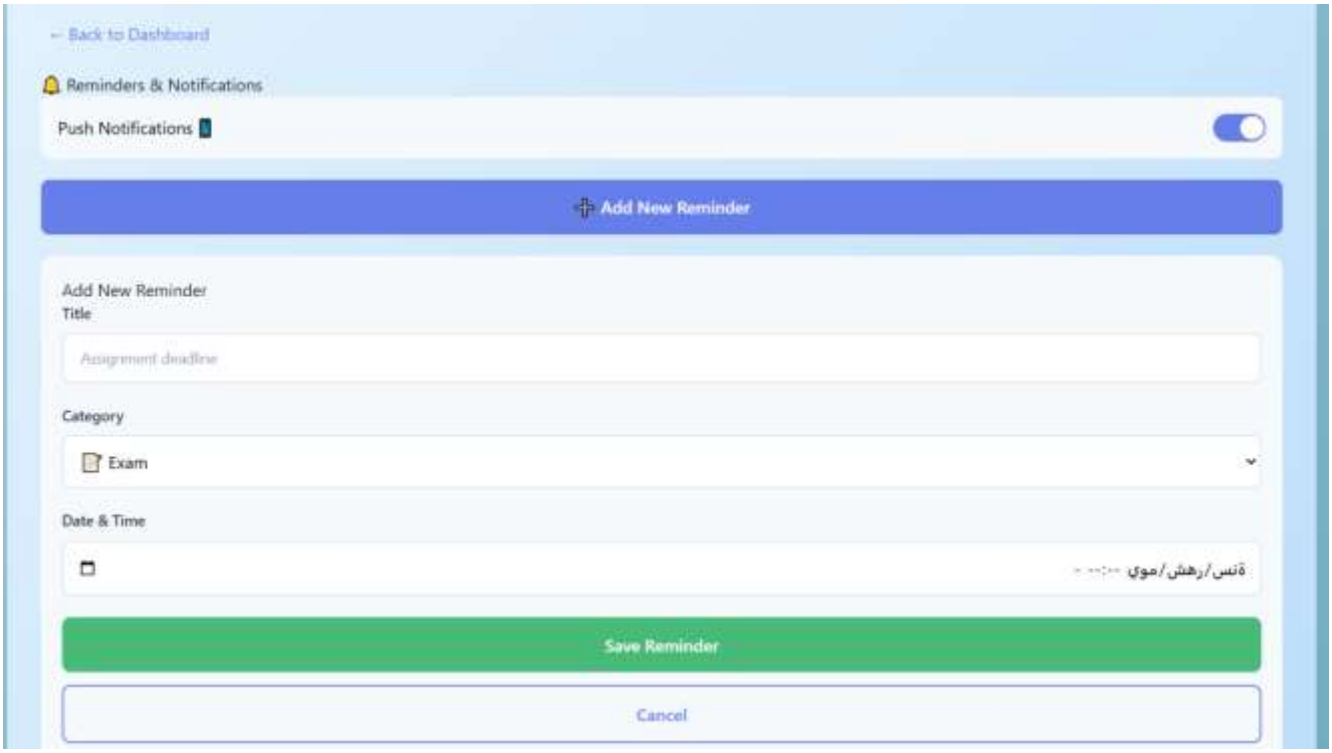



Figure 9: Add New Reminder Interface

### 4.3.4.4 Notification Settings Screen

Toggles push notifications and alert preferences.

### 4.3.5.4 Internship Status Tracking Screen

Displays application progress (Applied, Pending, Completed).

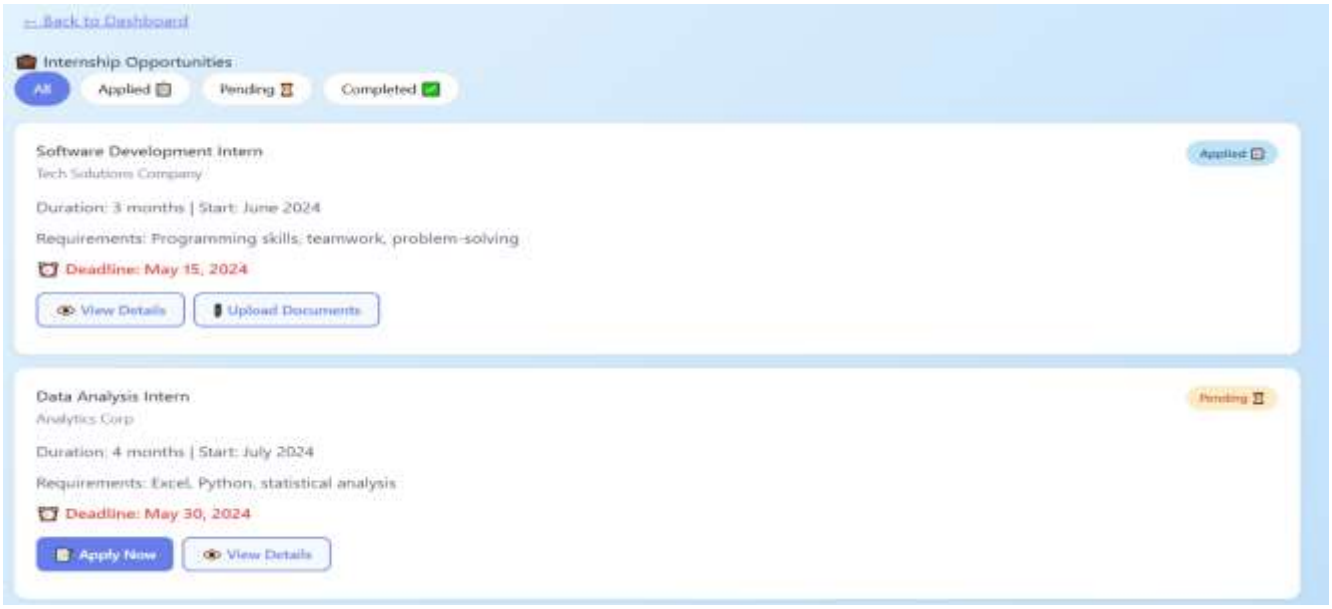



Figure 10: Internship Status Tracking Screen

### 4.3.5.5 class student

Displays Class Schedule

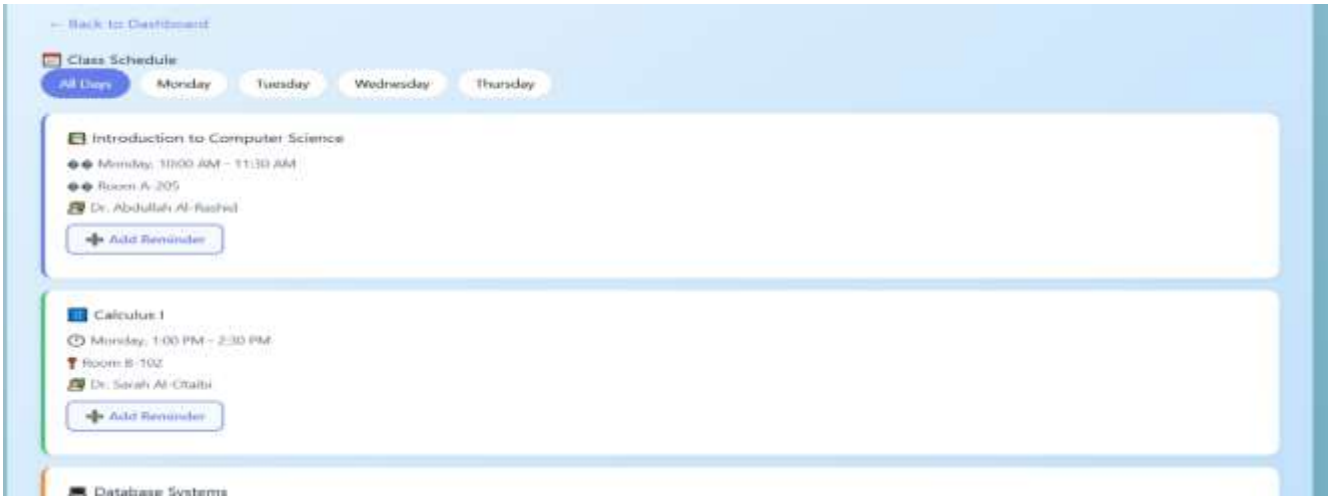



Figure 11: Class Schedule

### 4.3.6 University Announcements Interfaces

## 4.3.6.1 University Announcements List Screen

A list of all campus announcements.

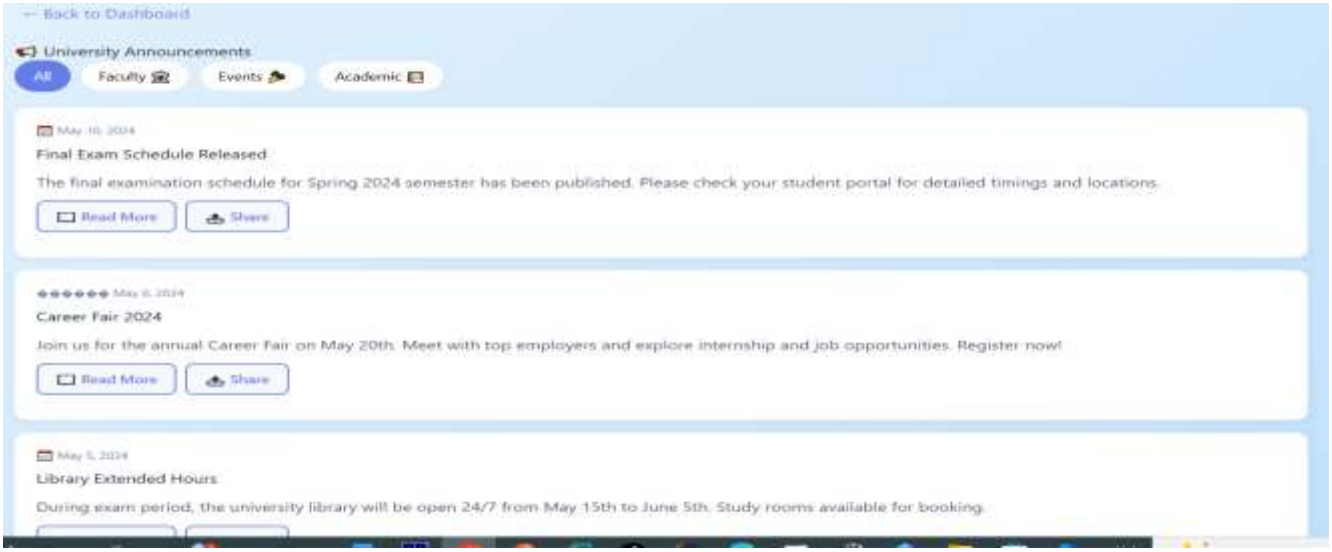



Figure 12: University Announcements List Screen

## 4.3.8 Profile & Account Interfaces

## 4.3.8.1 Profile Screen

Displays user information and academic profile.




Figure 13: Profile Screen

### 4.3.9 Settings & Preferences Interfaces

#### 4.3.9.1 Settings Screen

Central hub for managing all user preferences.

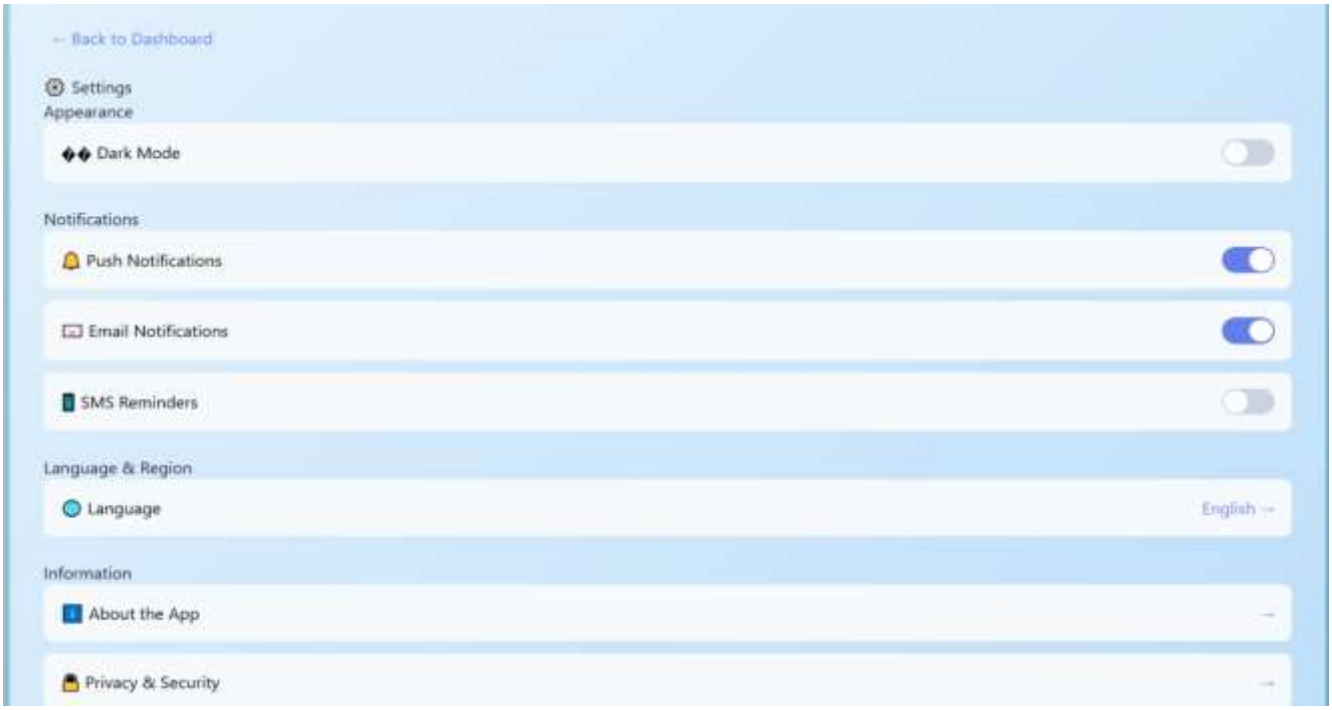



Figure 14: Settings Screen

### 4.3.10 Error & System Feedback Interfaces

Includes screens for:

- Invalid Email or Password
- No Internet Connection
- Schedule Not Synced
- Missing Required Fields

Each error uses clear colors, icons, and recovery instructions.




Figure 15: Error & System Feedback Screen Samples

# 5. System Architecture

This chapter provides a comprehensive architectural description of the WiseOwl system. It outlines the architectural approach, the global system structure, and detailed subsystem architecture. The purpose of this section is to establish a clear understanding of how system components interact, communicate, and support the functional and non-functional requirements defined in earlier chapters.

## 5.1 Architectural Approach

The architectural approach chosen for the WiseOwl system is a **modular, layered, and service-oriented architecture (SOA)**. It divides the system into well-defined layers to maximize maintainability, scalability, and separation of concerns.

The approach is based on the following principles:

### 1. Separation of Concerns

Each system layer (Presentation, Business Logic, Data Access, and Integration) performs specialized responsibilities, ensuring clean boundaries between components.

### 2. Scalability

The architecture supports incremental expansion—new modules (e.g., new academic features, Search-based feedback, advanced analytics) can be introduced without disrupting existing components.

### 3. Maintainability

A layered architecture improves code maintainability by isolating business rules from UI and database implementation.

### 4. Security

Access control, authentication, and data protection mechanisms are centralized at the business logic and integration layers.

### 5. Interoperability

The system integrates seamlessly with third-party services such as university authentication providers, cloud messaging platforms, and database services.

### 6. Technology Flexibility

Front-end (React, Angular, Flutter Web), backend (Node.js, Python, Java Spring), and database technologies (PostgreSQL, MySQL, or Firestore) can be selected independently.

This architectural approach ensures coherent system structure, efficient performance, and future evolution without major refactoring.

## 5.2 Overall Architectural Design

The WiseOwl platform follows a **four-layer architecture**, each layer responsible for a specific set of operations, data handling, and communication flows. These layers collectively form the backbone of the system.

### Layers Overview

1. **Presentation Layer**
    - Manages the user interface, navigation flow, and client-side validation.
    - Built using web technologies (HTML, CSS, JS Frameworks).
    - Communicates with backend services via REST APIs.
2. **Business Logic Layer**
    - Contains all application rules and workflows (e.g., handling schedules, reminders, internships).
    - Processes requests, applies validation, and enforces permissions.
    - Acts as a mediator between UI and data layers.
3. **Data Access & Persistence Layer**

- Manages all operations on persistent storage such as student profiles, schedules, reminders, and university announcements.
- Ensures ACID operations, indexing, transactions, and schema consistency.

4. **External Integration Layer**
   - Handles communication with external systems including:
     - University Authentication Services
     - Cloud Messaging Platforms
     - File Storage
     - Internship APIs (if applicable)

This layered structure ensures clean data flow and independent module evolution.

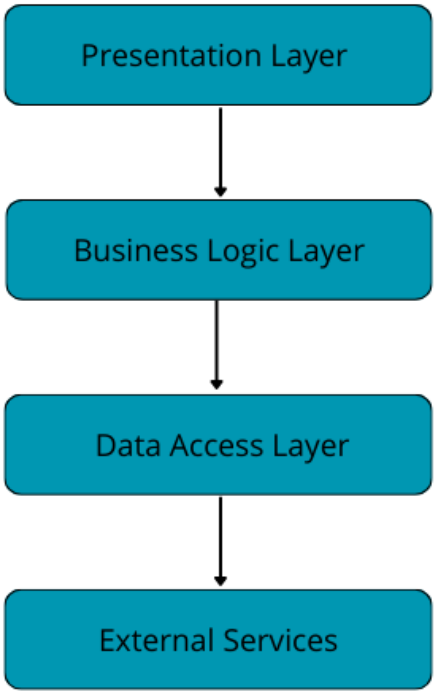



Figure 16: Layered Architecture Diagram

## 5.3 Subsystem Architecture

The WiseOwl system is decomposed into subsystems based on functional grouping. Each subsystem corresponds to one or more layers in the architecture and interacts through well-defined interfaces.

### 5.3.1 Presentation Layer

The Presentation Layer is responsible for all user-facing interactions. It manages UI rendering, user navigation, and client-side processing.

#### Responsibilities

- Rendering of dashboards, schedules, reminders, messaging screens, and profile pages
- Handling form submissions and client-side validations
- Sending requests to backend APIs
- Providing responsive layouts for desktop, tablet, and mobile devices
- Managing session state and UI language preferences

#### Technologies

- React.js / Angular / Vue.js / Flutter Web
- HTML5, CSS3, JavaScript
- Bootstrap / TailwindCSS

#### Subsystem Components

- Authentication UI
- Dashboard & Navigation Screens
- Schedule Viewer
- Reminders Module UI
- Internship Module UI
- Announcement Viewer
- Messaging UI
- Settings & Profile UI

### 5.3.2 Business Logic Layer

The Business Logic Layer contains the operational rules and workflows that define WiseOwl's behavior.

#### Key Responsibilities

- Enforcing business rules (e.g., schedule conflicts, reminder constraints)
- Processing user requests (create, update, delete operations)
- Applying authentication and authorization logic
- Managing state transitions (e.g., internship application statuses)
- Integrating with external APIs and cloud services
- Handling notification logic (push, email, alerts)

#### Core Services

- **Schedule Management Service**

- **Reminders & Notifications Service**
- **Internship Workflow Service**
- **Announcements Service**
- **Communication/Messaging Service**
- **User Profile & Settings Service**

These services ensure proper execution of all platform features.

### 5.3.3 Data Access & Persistence Layer

The Data Access Layer manages all operations on persistent data storage.

#### Responsibilities

- Executing CRUD operations
- Mapping database models to business objects
- Enforcing constraints and relationships
- Ensuring ACID compliance
- Managing backups, replication, and indexing

#### Database Options

- **SQL Databases:** PostgreSQL, MySQL
- **Cloud Databases:** Firebase Firestore
- **Storage:** AWS S3 / Firebase Storage for documents

#### Data Entities

- Users
- Class Schedules
- Reminders
- Internship Listings
- Internship Applications
- Messages
- Announcements
- Settings & Preferences

This layer ensures structured, secure, and optimized data management.

### 5.3.4 External Integration Layer

This layer manages all communication with external systems required by WiseOwl.

#### Responsibilities

- Handling authentication flows (OAuth, SSO, or University Identity Provider)
- Communicating with cloud messaging services (push notifications)

- Storing files and images via cloud storage
- Accessing third-party APIs for internships or university data

## External Integration Components

### 1. University Services APIs

Used for:

- Student verification
- Faculty information retrieval
- Accessing university announcements or calendar data
- SSO authentication (Shibboleth, LDAP, OAuth)

### 2. Firebase / Cloud Services

Used for:

- Real-time notifications (Firebase Cloud Messaging)
- Document and media storage (Firebase Storage / AWS S3)
- App analytics and crash monitoring
- Optional realtime chat or messaging support

### 3. Authentication & Cloud Messaging

Handles:

- User login sessions
- Token validation
- Push notifications for reminders, schedule changes, and updates
- Device registration for notification delivery

### API Integrations

| Integration Type | Example Provider | Purpose | Data Exchanged |
|---|---|---|---|
| **University SSO API** | LDAP / OAuth2 | Student & faculty authentication | User credentials, tokens |
| **University Services API** | University IT | Class info, announcements | Schedules, faculty info |
| **Cloud Storage API** | Firebase / AWS S3 | Document handling | Upload/download files |
| **Cloud Messaging API** | Firebase Cloud Messaging | Push notifications | Device tokens, alert payloads |
| **Internship External APIs** | Third-party companies | Internship listings (optional) | Internship details, application status |

# 6. Data Design

This chapter defines the data structures, entities, relationships, and database schema that support the WiseOwl platform. It ensures that all data stored, processed, and retrieved by the system is consistent, secure, and optimized for the workflows described in the SRS and SDS. The data design provides a foundation for backend implementation and ensures that data integrity is maintained across all modules.

## 6.1 Data Description

This section provides a high-level overview of the primary data entities used in the WiseOwl system. These entities represent all major concepts such as users, schedules, reminders, internships, announcements, and communication messages. The goal is to provide an overview before presenting the detailed data dictionary and the final structural schema.

### Data Description Overview

| Entity | Description | Key Attributes |
|---|---|---|
| **User** | Stores all user information (students, mentors, admins). | UserID, Name, Email, Role, LanguagePreference |
| **Schedule** | Represents weekly class schedules for students. | ScheduleID, CourseName, Day, Time, Location, Instructor |
| **Reminder** | User-created tasks or alerts tied to schedules or independent tasks. | ReminderID, Title, Date, Time, Priority, UserID |
| **Internship** | Internship postings from companies or university departments. | InternshipID, Title, Company, Description, Deadline |
| **InternshipApplication** | Tracks the progress of a student's application for an internship. | ApplicationID, InternshipID, UserID, Status |
| **Announcement** | University-wide messages or alerts. | AnnouncementID, Title, Category, PostedDate |
| **MessageThread** | Represents communication threads between users and departments. | ThreadID, UserID, Department, LastUpdated |
| **Message** | Individual messages in a thread. | MessageID, ThreadID, Sender, Content, Timestamp |
| **UploadedDocument** | Files uploaded for internships or other purposes. | DocumentID, UserID, FileURL, UploadedDate |
| **Settings** | User-specific preferences such as theme and notification preferences. | SettingsID, UserID, ThemeMode, NotificationsEnabled |

## 6.2 Data Dictionary

The Data Dictionary provides detailed descriptions of all fields in the database. It includes data types, field constraints, and data usage descriptions.

**Data Dictionary**

| Field Name | Data Type | Description | Constraints |
|---|---|---|---|
| **UserID** | UUID | Unique identifier for each user | Primary Key, Not Null |
| **FullName** | VARCHAR(100) | User's full name | Not Null |
| **Email** | VARCHAR(150) | User's email address | Unique, Not Null |
| **PasswordHash** | VARCHAR(255) | Hashed password | Not Null |
| **Role** | ENUM(Student, Mentor, Admin) | User type | Not Null |
| **LanguagePreference** | ENUM(English, Arabic) | Preferred UI language | Default = English |
| **ScheduleID** | UUID | Unique schedule identifier | Primary Key |
| **CourseName** | VARCHAR(100) | Course title | Not Null |
| **Day** | ENUM(Mon–Sun) | Day of the week | Not Null |
| **StartTime** | TIME | Class start time | Not Null |
| **EndTime** | TIME | Class end time | Not Null |
| **Location** | VARCHAR(100) | Class location | Nullable |
| **Instructor** | VARCHAR(100) | Instructor name | Nullable |
| **ReminderID** | UUID | Unique reminder identifier | Primary Key |
| **Title** | VARCHAR(150) | Reminder title | Not Null |
| **Date** | DATE | Reminder date | Not Null |
| **Time** | TIME | Reminder time | Not Null |
| **Priority** | ENUM(Low, Medium, High) | Reminder importance | Default = Medium |
| **InternshipID** | UUID | Unique internship ID | Primary Key |
| **Company** | VARCHAR(150) | Internship provider | Not Null |
| **Description** | TEXT | Detailed information | Not Null |
| **Deadline** | DATE | Deadline for application | Not Null |
| **ApplicationID** | UUID | Unique application identifier | Primary Key |
| **Status** | ENUM(Applied, Pending, Completed) | Application status | Default = Pending |
| **AnnouncementID** | UUID | Unique announcement identifier | Primary Key |
| **Category** | ENUM(Faculty, Events, Alerts) | Announcement type | Not Null |
| **PostedDate** | DATE | Announcement date | Not Null |
| **ThreadID** | UUID | Communication thread ID | Primary Key |
| **Department** | VARCHAR(100) | Target department | Not Null |
| **MessageID** | UUID | Message identifier | Primary Key |

| | | | |
|---|---|---|---|
| **Sender** | VARCHAR(100) | Message sender | Not Null |
| **Content** | TEXT | Message content | Not Null |
| **Timestamp** | DATETIME | Time message was sent | Not Null |
| **DocumentID** | UUID | Document record ID | Primary Key |
| **FileURL** | VARCHAR(255) | Cloud storage URL | Not Null |
| **UploadedDate** | DATETIME | Upload timestamp | Not Null |
| **SettingsID** | UUID | Settings record ID | Primary Key |
| **ThemeMode** | ENUM(Light, Dark) | User theme preference | Default = Light |
| **NotificationsEnabled** | BOOLEAN | Whether notifications are enabled | Default = True |

## 6.3 Database Schema

This section describes the logical and physical structure of the database. It includes both the Entity-Relationship Diagram (ERD) and the Relational Schema, which define how tables relate to each other and how data flows throughout the system.

### Entity-Relationship Diagram (ERD)

The ERD illustrates the major entities in the WiseOwl system and the relationships between them.




Figure 17: ER Diagram

**Key Relationships:**

- **User —< Schedule** (One user can have many schedule entries)
- **User —< Reminder** (One user can create multiple reminders)
- **User —< InternshipApplication >— Internship** (Many-to-many via applications)
- **Announcement** is standalone but viewable by all users
- **User —< Thread —< Message** (Each user may have multiple threads and messages)
- **User —< UploadedDocument**

- **User —< Settings** (One-to-one)

### Relational Schema

The relational schema provides the SQL-level structure of the database, defining tables, primary keys, and foreign keys.

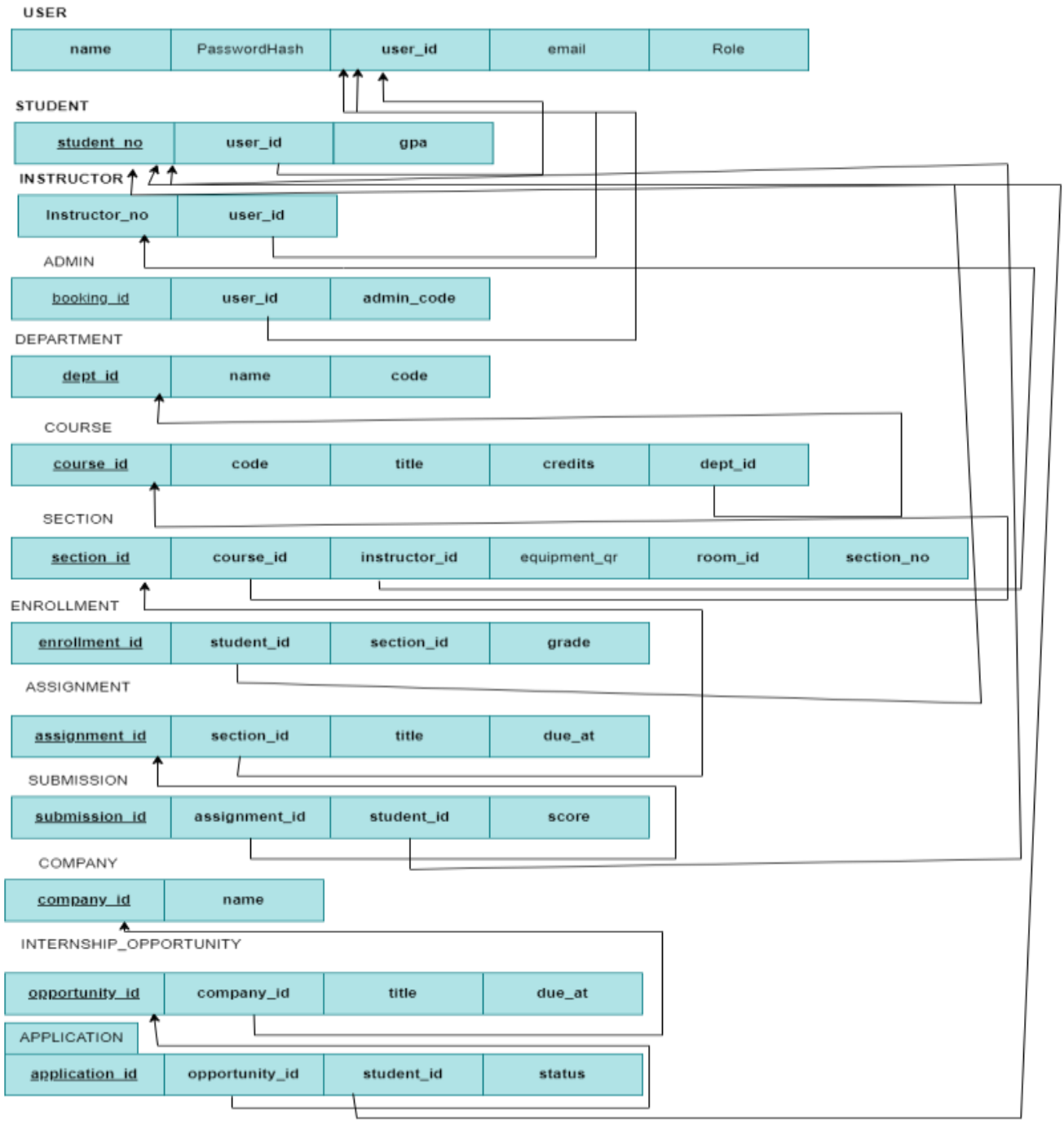



Figure 18: Relational Schema

This schema ensures referential integrity, supports fast queries, and aligns with the platform's workflows.

## 7. Component Design

This chapter describes the major components that form the WiseOwl system. Each component is defined in terms of its responsibilities, inputs, outputs, constraints, and interactions with other parts of the system. The objective is to ensure clear architectural decomposition, maintainability, and traceability to the functional requirements defined in the SRS.

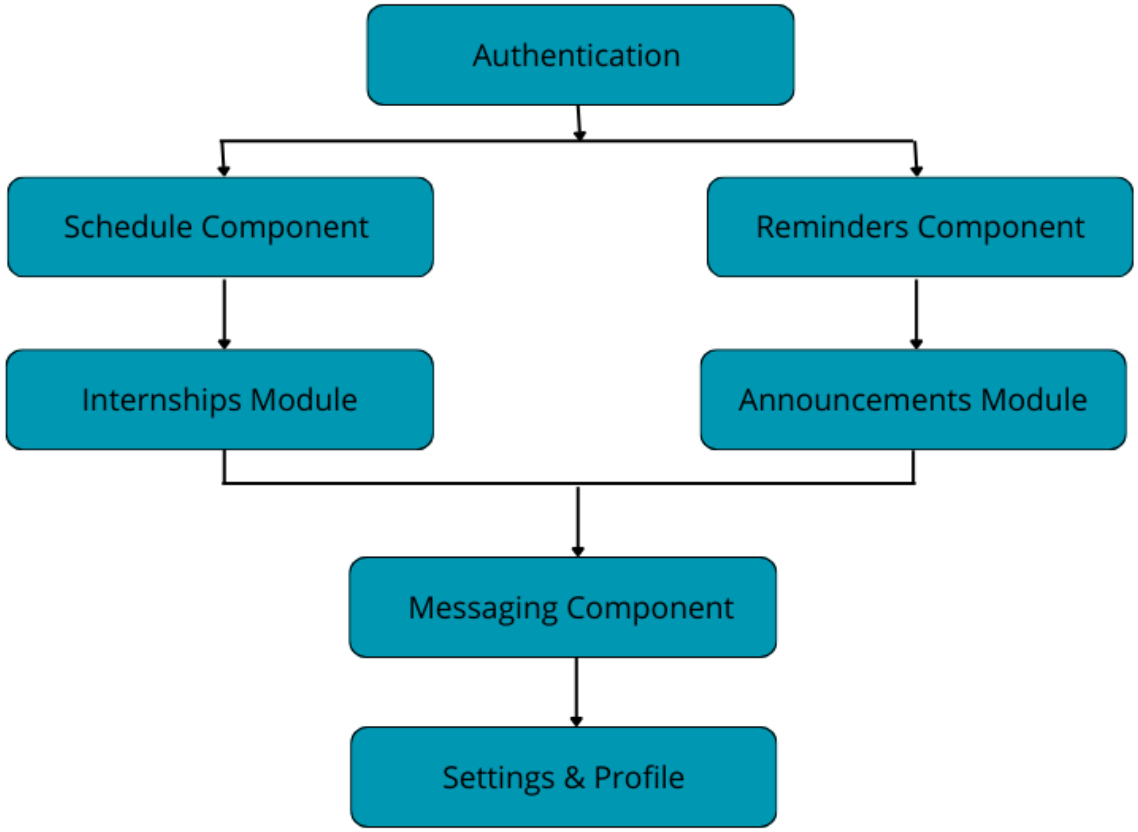



Figure 19: High-Level Component Diagram

This diagram illustrates the major subsystems of WiseOwl, including Authentication, Schedule Management, Reminders, Internships, Announcements, Messaging, and Settings. It also highlights the interactions between the Presentation Layer, Business Logic Layer, Data Layer, and External Services.

### 7.1 Common Components

These components are foundational and support multiple modules within the WiseOwl system.

### 7.1.1 Authentication & Session Management

The Authentication component manages user identity verification, login sessions, credential management, and token lifecycle.

#### Responsibilities

- User login (email/password, optional SSO)
- Password hashing and verification
- Token creation (JWT or session tokens)
- Token expiration and renewal
- Logout and session invalidation
- Multi-language session handling

#### Inputs

- Login credentials
- Password reset requests
- Token refresh requests

#### Outputs

- Access tokens
- Session metadata
- Authentication success/error responses

#### Interactions

- Works with University SSO API (optional)
- Communicates with User Database
- Provides authentication status to all core modules

### 7.1.2 Authorization & Roles

Authorization ensures that only permitted users can access specific operations. WiseOwl uses **Role-Based Access Control (RBAC)**.

#### Supported Roles

- **Student**
- **Mentor**
- **Administrator**

#### Responsibilities

- Enforcing access rules for modules and actions
- Validating permissions based on role

- Supporting future role expansion

**Roles and Permissions Matrix**

| Feature / Action | Student | Mentor | Admin |
|---|---|---|---|
| **View Schedule** | Yes | Yes | Yes |
| **Edit Schedule** | No | No | Yes |
| **Create Reminders** | Yes | No | Yes |
| **View Announcements** | Yes | Yes | Yes |
| **Manage Announcements** | No | No | Yes |
| **View Internships** | Yes | Yes | Yes |
| **Apply for Internship** | Yes | No | No |
| **Review Internships** | No | Yes | Yes |
| **Messaging Departments** | Yes | Yes | Yes |
| **Manage Users** | No | No | Yes |

### 7.1.3 Notification & Messaging

This component handles system notifications across modules.

**Responsibilities**

- Sending push notifications for important deadlines
- Messaging students and departments
- Managing notification preferences
- Storing messages, timestamps, and thread history

**Message Types**

- System alerts (e.g., new announcements)
- Reminders (high-priority deadlines)
- Internship updates (status changes)
- Messages between users and departments

## 7.2 Core WiseOwl Components

These components make up the essential functional modules of the WiseOwl platform.

### 7.2.1 Student Management

This component manages all student-related information and operations.

**Responsibilities**

- Creating and updating student profiles
- Storing academic details (faculty, major, level)

- Linking schedules, reminders, internships, and settings
- Handling language preferences

**Primary Data Entities**

- User
- Schedule
- Reminders
- Settings

### 7.2.2 Schedule Management

This component manages class schedules.

**Responsibilities**

- Storing weekly course schedules
- Handling conflicts in time slots
- Allowing admins to assign class schedules
- Displaying schedule in weekly and daily formats

**Inputs**

- Course details
- Day/time slot
- Instructor information

**Outputs**

- Timetable layout
- Upcoming class notifications

**Data Types for Schedule & Reminders**

| Attribute | Data Type | Description |
|---|---|---|
| **CourseName** | String | Course title |
| **Day** | Enum | Weekday |
| **StartTime** | Time | Class start time |
| **EndTime** | Time | Class end time |
| **Location** | String | Classroom / hall |
| **Instructor** | String | Lecturer’s name |
| **ReminderTitle** | String | Reminder text |
| **ReminderPriority** | Enum | Low/Medium/High |

### 7.2.3 Reminders Management

Handles academic, personal, and schedule-based reminders.

#### Responsibilities

- Creating, editing, and deleting reminders
- Categorizing reminders by priority
- Sending push notifications
- Integrating reminders with schedules

#### Output

- Notification events
- Reminder lists

### 7.2.4 Internship Management

Manages internship opportunities and student applications.

#### Responsibilities

- Displaying internship listings
- Handling applications and status updates
- Storing documents such as CVs and transcripts
- Tracking application progress

#### Internship Status Categories

| Status | Description |
|---|---|
| **Applied** | Initial submission |
| **Pending** | Under review |
| **Completed** | Application approved or internship concluded |

### 7.2.5 Announcements Management

Handles distribution and display of university announcements.

#### Responsibilities

- Managing announcement categories
- Publishing new announcements
- Filtering announcements by type
- Displaying announcement details

#### Announcement Categories

| Category | Description |
|---|---|
| **Faculty** | Faculty-related updates |
| **Events** | Campus or academic events |

| | |
|---|---|
| **Alerts** | Urgent updates or warnings |

### 7.2.6 Communication / Messaging

Supports messaging between users and university departments.

#### Responsibilities

- Creating message threads
- Sending and receiving messages
- Handling file attachments
- Storing message history

#### Message Types

- Student to department
- Mentor to department
- Admin to department

## 7.3 Supporting Components

These components enhance reliability, personalization, and maintainability of the WiseOwl system.

### 7.3.1 Settings & Configuration

Manages user preferences and application configuration.

#### Responsibilities

- Managing theme (Light/Dark)
- Language preferences
- Notification preferences
- User profile settings

#### User Settings and Configuration Parameters

| Setting | Type | Description |
|---|---|---|
| **ThemeMode** | Enum | Light or Dark |
| **Language** | Enum | Arabic / English |
| **NotificationEnabled** | Boolean | Enable/disable alerts |
| **AutoSync** | Boolean | Controls automatic data sync |

### 7.3.2 Backup & Synchronization

Ensures data consistency between client and server.

**Responsibilities**

- Automatic backup of critical data
- Syncing schedules and reminders
- Offline support for essential data
- Recovery mechanisms after connection loss

### 7.3.3 Logging & Monitoring

Tracks system events, errors, and performance metrics.

**Responsibilities**

- Error logging
- Usage analytics
- Performance monitoring
- Security and audit logs

**Component Responsibilities**

| Component | Responsibilities |
|---|---|
| **Authentication** | Login, session tokens, password resets |
| **Schedule Management** | Store and display schedules |
| **Reminders** | Manage reminders and alerts |
| **Messaging** | Manage conversations |
| **Internships** | Track applications |
| **Announcements** | Publish and manage announcements |
| **Settings** | Manage user preferences |

**Component Constraints**

| Component | Constraints |
|---|---|
| **Authentication** | Must comply with institutional SSO requirements |
| **Messaging** | Must support high-volume interactions |
| **Schedule** | Must avoid overlapping times |
| **Reminders** | Notifications must be timely and reliable |
| **Internships** | Document uploads must be secure |
| **Logging** | Logs must not contain sensitive data |

## 8. Detailed System Design

This chapter provides the complete internal design of the WiseOwl system, including classifications, definitions, user interaction flows, resource requirements, processing logic, interfaces, and subsystem-level sequence diagrams. It defines how each module works internally and how the system behaves during execution.

## 8.1 Classification

The WiseOwl system is classified as a **web-based academic management platform** designed for higher education institutions. It supports student schedule tracking, reminders, internships, announcements, messaging, and personal settings.

### Classification by Functional Domain

- **Academic Scheduling System** (Course schedules, class tracking)
- **Student Productivity System** (Reminders and notifications)
- **Career Development System** (Internship listings and applications)
- **Communication System** (Messaging with university departments)
- **Information Delivery System** (Announcements and alerts)
- **User Preference System** (Profiles, settings, and configurations)

### Classification by Architecture Type

- **Layered Architecture** (Presentation, Business Logic, Data, Integration)
- **Service-Oriented Components**
- **RESTful API–based Interactions**
- **Database-oriented Persistence Layer**

## 8.2 Definitions

This section defines key terms and internal system concepts used throughout the design.

### Key Definitions

| Term | Definition |
|---|---|
| Subsystem | A functional module that performs a specific set of tasks (e.g., authentication, reminders). |
| Sequence Diagram | A behavioral UML diagram showing interactions over time. |
| User Session | A temporary state created after successful authentication. |
| Notification Event | A system-triggered alert such as a reminder or internship update. |
| Data Entity | A structured object stored in the database (User, Schedule, Reminder). |
| Component Interface | The exposed operations a module provides to others. |
| Processing Flow | Logical steps executed during an operation (e.g., scheduling a reminder). |

## 8.3 User Interactions & Use Flows

This section describes how users interact with the system’s major workflows. These user flows guide the sequence diagrams in Section 8.7.

### Examples of Key Use Flows

1. **Login Flow**
   - User enters credentials → system validates → session created → dashboard displayed.
2. **View Weekly Schedule**
   - User opens Schedule module → system retrieves user's schedule → renders weekly calendar.
3. **Create New Reminder**
   - User selects reminder type → enters date/time → system stores reminder → notification scheduled.
4. **Apply for Internship**
   - User views internship → submits documents → system creates application → status tracked.
5. **View Announcement**
   - User selects category → system fetches announcement → displays full details.
6. **Send a Message to a Department**
   - User opens Messaging module → selects department → composes message → system stores and delivers it.
7. **Modify Profile Settings**
   - User edits settings → system validates → saved in database → applied to UI immediately.

These flows ensure clarity during implementation and testing.

## 8.4 Resource Requirements

This section defines expected system resources to support reliable processing and storage.

### System Resource Requirements

| Resource Type | Requirement | Notes |
|---|---|---|
| **CPU** | 2–4 vCPUs | For handling concurrent backend requests |
| **Memory** | 4–8 GB RAM | Ensures smooth processing for up to 500 users |
| **Storage** | 50–200 GB | Depends on volume of uploaded documents |
| **Network** | ≥ 50 Mbps | Ensures smooth web interactions |
| **Database** | SSD-backed storage | Required for fast I/O operations |
| **Cloud Messaging** | Firebase / AWS SNS | Required for notifications |
| **Authentication** | OAuth2 / SSO Provider | Must support institutional authentication |

## 8.5 Processing Logic

Processing logic defines how the system manipulates data and executes procedures.

### General Processing Patterns

- **Input Validation:** All user inputs are validated in UI and server layers.

- **Business Rules Enforcement:** Applied by the Business Logic Layer.
- **Error Handling:** Standard error responses for invalid operations.
- **Data Persistence:** All operations pass through the Data Access Layer.
- **Event Triggering:** Creating reminders triggers notification scheduling.
- **Cache Management:** Frequently accessed items (schedules, announcements) may be cached.

### Examples of Processing Logic

1. **Login Processing Logic**
   - Validate email/password → retrieve user record → verify password → return token.
2. **Reminder Creation Logic**
   - Validate date/time → insert record → schedule notification → confirm creation.
3. **Internship Application Logic**
   - Validate documents → create application record → notify user → update status.
4. **Settings Update Logic**
   - Validate settings → update database → refresh UI preferences.

## 8.6 Interfaces / Exports

The WiseOwl system exposes REST API interfaces to support integration between the UI and backend services.

### Interface Types

- **Authentication APIs**
- **Schedule APIs**
- **Reminder APIs**
- **Internship APIs**
- **Announcement APIs**
- **Messaging APIs**
- **Profile & Settings APIs**

### Outputs (Exports)

- JSON responses
- Notification events
- Status codes (200, 400, 404, 500)

These interfaces ensure modular and scalable interactions.

## 8.7 Subsystem Detailed Design

This section provides subsystem-level sequence diagrams that illustrate operational details.

### 8.7.1 Authentication & User Management Subsystem

Handles login, registration, password reset, and session creation.

**Flow Description**

1. User enters login credentials
2. System validates fields
3. Backend retrieves user record
4. Password is verified
5. Token/session is created
6. User is redirected to dashboard

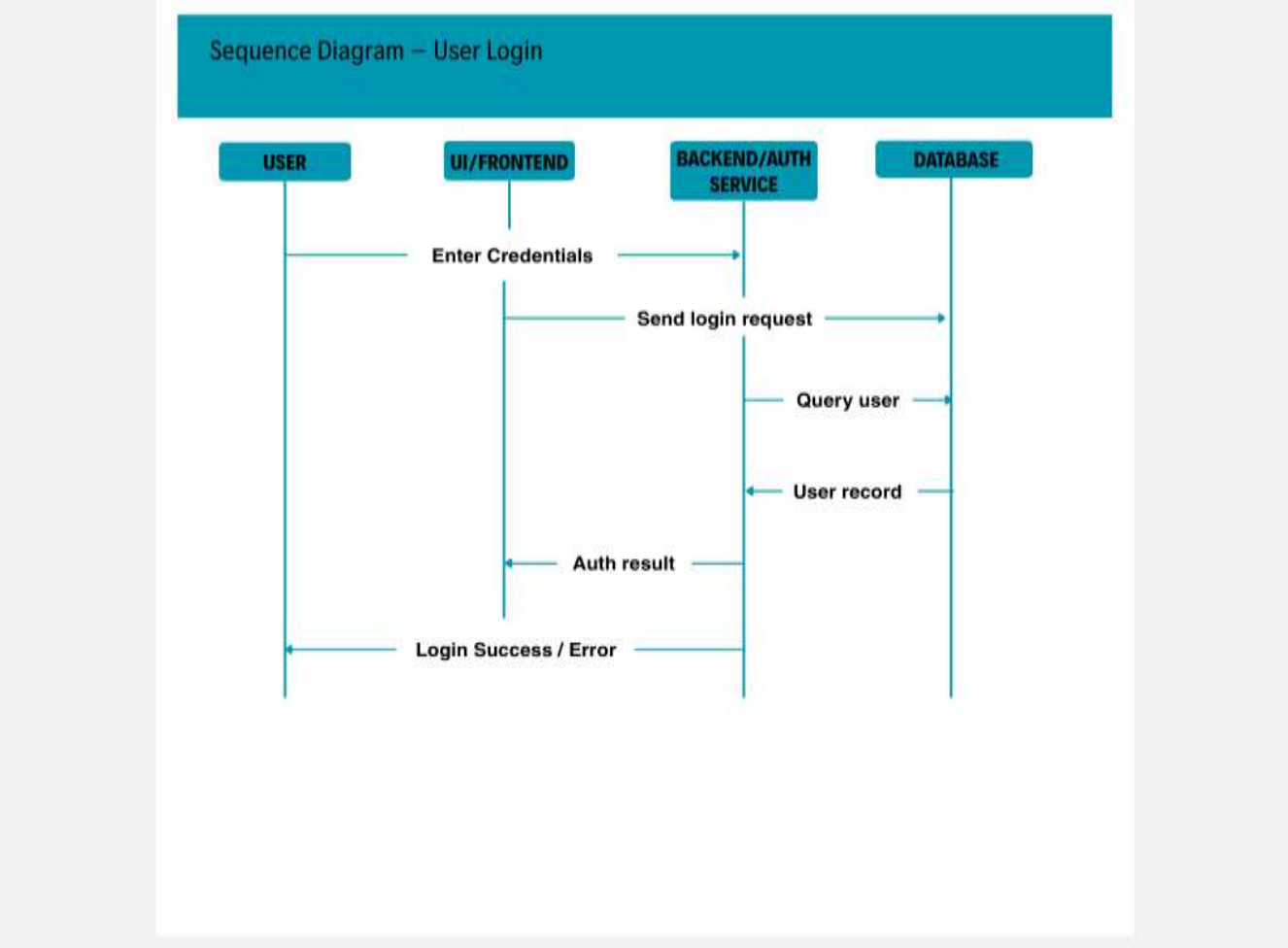



Figure 20: Sequence Diagram – Login

### 8.7.2 Schedule & Reminders Subsystem

Manages class schedules and reminder creation.

### Flow Description

1. User requests schedule view
2. System retrieves schedule from database
3. User selects a class
4. User adds a reminder
5. System stores reminder and schedules notifications

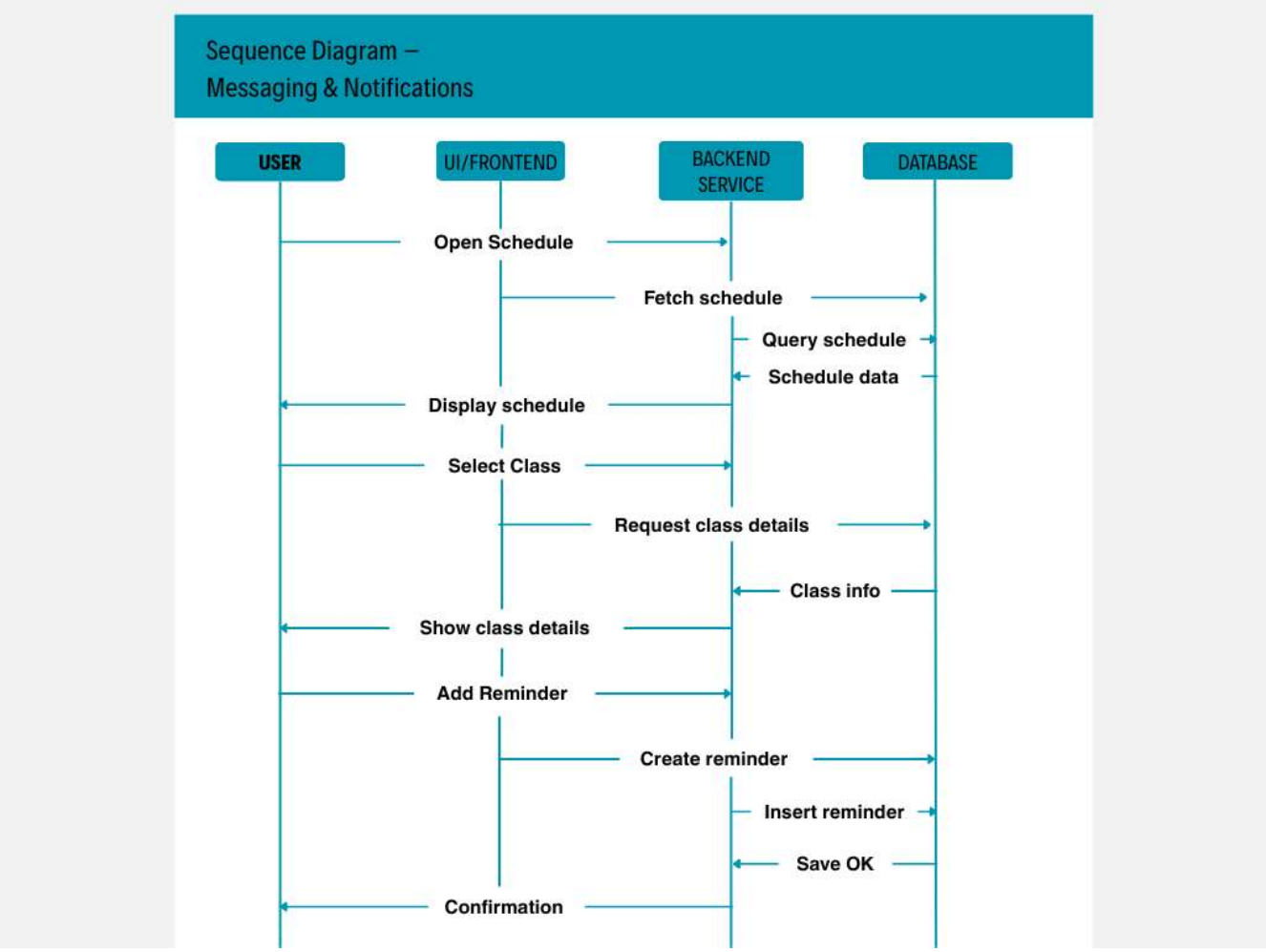



Figure 21: Sequence Diagram – Schedule Management

## 8.7.3 Internship Tracking Subsystem

Handles internship listings, applications, and status changes.

### Flow Description

1. User views internship listing
2. System retrieves details
3. User submits application
4. Documents are uploaded

5. Application record created
6. Status is tracked

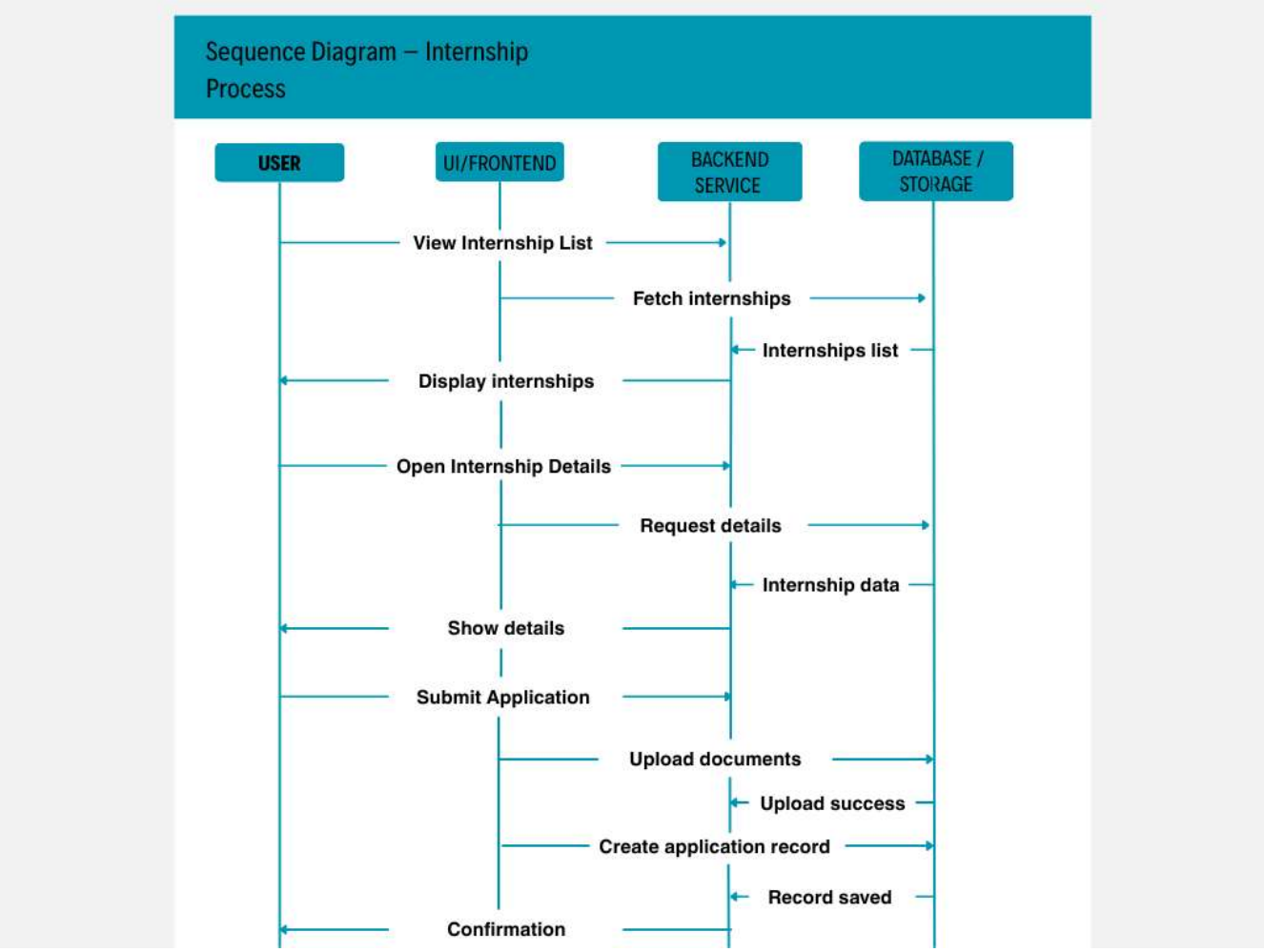



Figure 22: Sequence Diagram – Internship Process

### 8.7.4 Announcements Subsystem

Responsible for publishing, viewing, and filtering university announcements.

**Flow Description**

- User selects announcement category
- System fetches relevant announcements
- User views detailed announcement
- System logs view for analytics

This subsystem ensures timely access to academic and campus alerts.

### 8.7.5 Messaging Subsystem

Handles communication between users and departments.

**Flow Description**

1. User selects a department
2. System fetches message thread
3. User sends message
4. System stores message and notifies department
5. Department replies
6. User receives notification

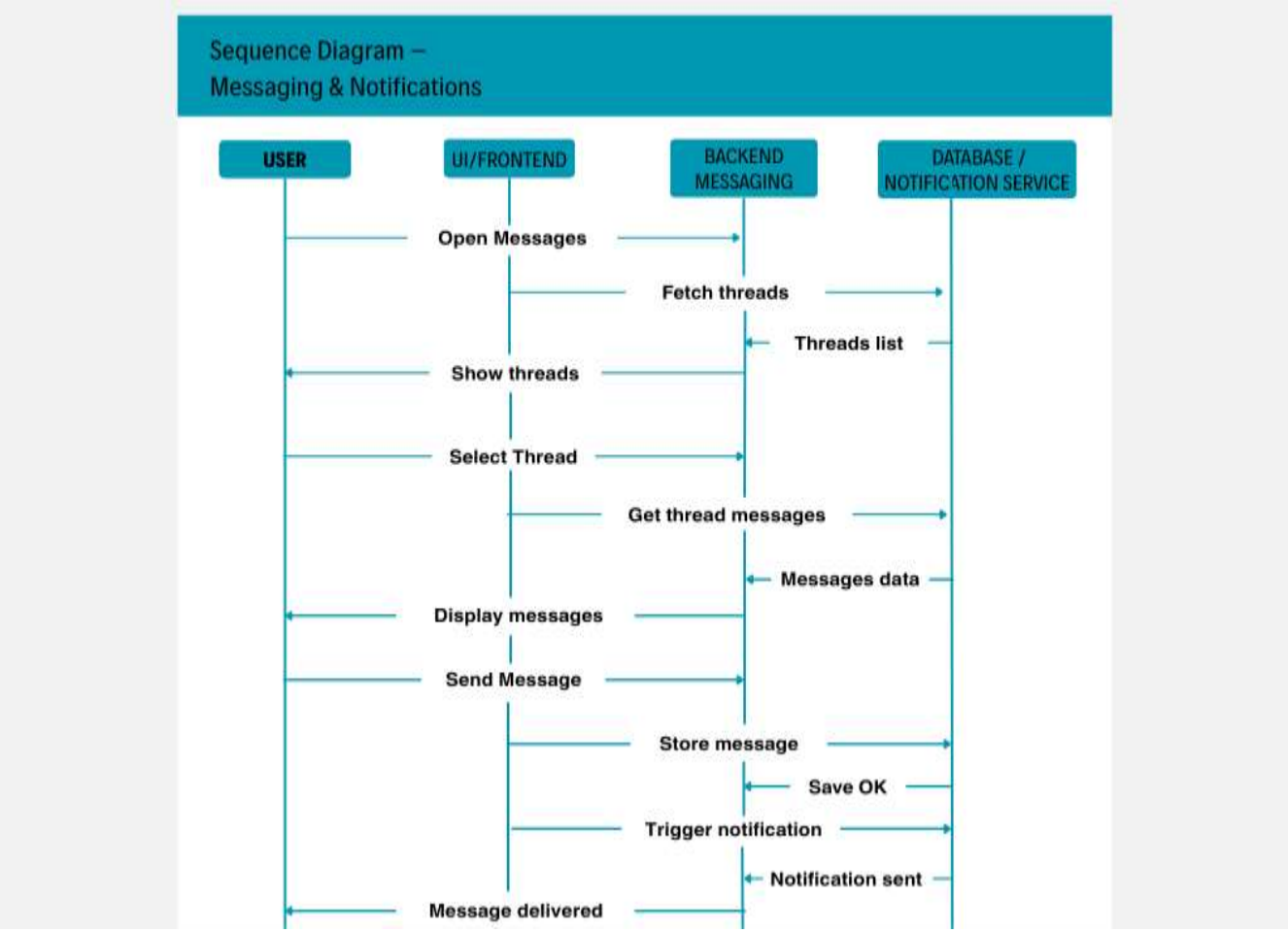



Figure 23: Sequence Diagram – Messaging & Notifications

### 8.7.6 Profile & Settings Subsystem

Manages user preferences such as language, notifications, and theme mode.

**Flow Description**

1. User opens Settings
2. System loads current preferences
3. User updates preferences
4. System validates and saves changes
5. Updated settings applied to UI

This subsystem ensures a personalized user experience.

### 8.8 Responsibilities

This section summarizes the main responsibilities of the core subsystems of the IAU WiseOwl platform. The Authentication & User Management subsystem is responsible for registering users, verifying their identities, enforcing authentication and authorization rules, and managing user roles, profiles, and active sessions. The Schedule & Reminders subsystem is responsible for storing and presenting students' weekly class schedules, handling schedule-related validation (such as time conflicts), and managing academic and personal reminders, including scheduling notification events for upcoming classes, deadlines, and tasks. The Internship Tracking subsystem is responsible for presenting internship listings, managing student applications, handling document uploads, and tracking application status across states such as applied, pending, and completed. The Announcements subsystem is responsible for publishing, categorizing, and displaying university announcements, and supporting filtering by category so that students can quickly find relevant updates. The Messaging subsystem is responsible for enabling structured communication between students, faculty, and departments by managing message threads, storing message history, and coordinating the delivery of system and department responses. Finally, the Profile & Settings subsystem is responsible for maintaining user-specific preferences and configuration (such as language, theme mode, and notification settings), as well as profile details that personalize the WiseOwl experience for each user.

### 8.9 Constraints

The design and implementation of the IAU WiseOwl platform are subject to several technical and non-technical constraints specific to a mobile, university-centered environment. All subsystems must comply with IAU's security and privacy policies, including secure authentication mechanisms, encrypted communications over HTTPS/TLS, and proper protection of sensitive student, schedule, internship, and messaging data. The mobile-first nature of WiseOwl constrains the user interface and processing logic to operate efficiently on Android and iOS devices with varying hardware capabilities and network conditions, which drives the need for lightweight screens, optimized queries, and responsive layouts. The chosen technology stack—cross-platform mobile framework, RESTful backend services, cloud-based storage and messaging, and a relational database—constrains integration patterns, deployment options, and error-handling strategies, and requires adherence to the capabilities and limits of these platforms (such as API rate limits, authentication flows, and storage models). Performance and scalability constraints require that routine operations (such as loading schedules, fetching announcements, retrieving reminders, and updating internship status) complete within acceptable response times under peak academic load, which affects caching strategies, indexing, and the separation of responsibilities across subsystems. In addition, availability and reliability constraints, including backup and recovery requirements, necessitate robust error handling, transaction integrity at the data layer, and periodic backups so that user data (especially schedules, reminders,

internship records, and messages) can be recovered after failures. Institutional policies and applicable regulations further constrain how long data and logs are retained, how access control rules are enforced across user roles (student, faculty, administrator), and how audit trails are generated and protected for sensitive administrative and academic operations.

### 8.10 Composition

The IAU WiseOwl system is composed of a set of loosely coupled but strongly coordinated subsystems, each encapsulating a specific group of responsibilities while sharing a common architectural backbone. At the presentation layer, the mobile client provides the user interface for authentication, dashboards, schedule views, reminders, internship tracking, announcements, messaging, and profile/settings screens; it communicates with backend services through well-defined RESTful APIs using JSON as the standard data exchange format. The application (business logic) layer is internally organized into modules that correspond to the Authentication & User Management, Schedule & Reminders, Internship Tracking, Announcements, Messaging, and Profile & Settings subsystems; these modules implement the core workflows, enforce business rules, and coordinate interactions among features such as reminders, notifications, and internship status changes. All subsystems share a common data access and persistence layer that manages a centralized database for structured entities (users, schedules, reminders, internships, announcements, messages, and settings) and, where applicable, cloud storage for uploaded documents and media. An external integration layer connects WiseOwl to supporting services such as authentication providers, cloud messaging platforms for push notifications, and any university APIs used for institutional data. Cross-cutting concerns—including logging, monitoring, error handling, security enforcement, and backup/synchronization—are implemented as shared services invoked by multiple subsystems. This composition allows WiseOwl to evolve incrementally (for example, adding new academic features or extending analytics and messaging capabilities) without disrupting the overall architecture, while maintaining clear boundaries between subsystems and preserving maintainability, scalability, and reliability.

## 9. Additional Design Features

This chapter presents the additional design elements that enhance system performance, security, error resilience, localization, maintainability, and extensibility. These features ensure that WiseOwl remains reliable, scalable, and adaptable throughout its lifecycle.

### 9.1 Performance & Scalability

The WiseOwl system is engineered to support smooth performance under peak academic loads and maintain fast response times even with increased user activity.

### Performance Objectives

- Maintain response times under **300 ms** for 90% of requests.
- Support concurrent logins during high-traffic periods (e.g., start of semester).
- Efficiently handle large volumes of reminders, schedules, messages, and announcements.
- Provide stable performance across devices and network conditions.

### Scalability Principles

- **Horizontal scaling:** Add more servers or containers as user load increases.
- **Database indexing:** Optimize queries for schedules, reminders, and messaging.
- **Caching:** Cache frequently accessed data such as announcements and schedules.
- **Cloud-based storage:** Offload document storage from main database.

### Prototype Usability Metrics

| Metric | Target Value | Description |
|---|---|---|
| **Average Page Load Time** | ≤ 2 seconds | Measures time to render UI components |
| **API Response Time** | ≤ 300 ms | Time for backend to respond to requests |
| **Maximum Concurrent Users** | 500+ | Scalable based on server configuration |
| **Error Rate** | < 1% | Acceptable failure threshold |
| **Notification Delivery Time** | ≤ 5 seconds | Push message delivery speed |
| **Uptime** | ≥ 99.5% | Ensures service availability |

### System Availability SLA

| SLA Tier | Availability | Description |
|---|---|---|
| **Gold** | 99.9% | Near-continuous availability |
| **Silver** | 99.5% | Standard academic platform availability |
| **Bronze** | 99.0% | Minimum acceptable availability |

## 9.2 Security Architecture

Security is a core requirement of the WiseOwl system. The design includes multiple layers of protection to safeguard user information and system assets.

### Security Measures

#### 1. Authentication & Authorization

- Role-Based Access Control (RBAC)
- Multi-factor authentication (optional)
- Token-based sessions (JWT)

**2. Data Security**

- Password hashing (bcrypt/argon2)
- Encrypted storage of critical data
- HTTPS/TLS enforced across all endpoints

**3. Network Security**

- Firewall and IP filtering
- Request throttling to prevent brute-force attacks

**4. Application Security**

- Input validation and sanitization
- Prevention of XSS, SQL injection, CSRF
- Secure file uploads for internship documents

**5. Logging & Monitoring**

- Event logging for authentication attempts
- Audit trails for sensitive actions

The system architecture ensures compliance with institutional security policies and modern cybersecurity standards.

### 9.3 Error Handling & Fault Recovery

Error handling strategies ensure consistent user experience, while fault recovery measures preserve system integrity in case of failures.

#### Error Handling Approach

- **User-friendly messages** for all common errors.
- Categorization of errors into:
    - Input errors
    - Authentication errors
    - Server-side errors
    - Messaging failures
    - Network disconnections
- Automatic retries for network-sensitive operations.

#### Fault Recovery Strategies

- Database rollback for failed transactions.
- Scheduled backups with point-in-time recovery.
- Auto-reconnect for messaging and sync operations.
- Redundant storage for internship documents.
- Failover mechanisms for cloud services.

**Backup and Recovery Protocol**

| Component | Backup Frequency | Recovery Method | Notes |
|---|---|---|---|
| **User Data** | Daily | PITR (Point-in-Time Recovery) | Stored on cloud backup |
| **Schedules** | Hourly | Transaction log replay | High change frequency |
| **Reminders** | Daily | Automated restore | Includes notifications |
| **Messages** | Every 6 hours | Snapshot restore | Ensures communication continuity |
| **Announcements** | Daily | Full backup | Low update frequency |
| **Internship Documents** | Real-time | Cloud storage redundancy | FT (Fault-tolerant storage) |

## 9.4 Localization (Arabic / English)

Localization ensures that WiseOwl supports both English and Arabic languages, providing a seamless bilingual experience.

### Localization Features

- Language toggle in settings.
- Automatic RTL/LTR UI flipping.
- Localized date/time formats.
- Translated system messages, labels, and placeholders.
- Multilingual notifications for reminders and announcements.

### Localization Specifications

| Feature | English (LTR) | Arabic (RTL) | Notes |
|---|---|---|---|
| **UI Direction** | Left-to-right | Right-to-left | Automatic mirroring |
| **Typography** | Latin fonts | Arabic fonts | Ensures readability |
| **Date Format** | MM/DD/YYYY | DD/MM/YYYY | Localized by region |
| **Notifications** | English text | Arabic text | Based on preference |
| **Form Labels** | English | Arabic | Fully translated |
| **Error Messages** | English | Arabic | Context-sensitive |

Localization is applied dynamically to ensure consistency across the entire system.

## 9.5 Maintainability & Extensibility

The system is designed for long-term maintainability and easy extensibility.

### Maintainability Design Principles

- Modular component-based architecture
- Clear API boundaries
- Consistent naming conventions
- High code readability
- Logging for debugging and auditing
- Standardized error handling blocks

### Extensibility Features

- Easy addition of new components (e.g., attendance tracking)
- Plug-in model for future integrations
- Expandable settings and configurations
- Flexible notification engine for new event types
- Scalable architecture supporting future mobile apps

### Prototype Evaluation & Testing Checklist

| Category | Evaluation Criteria | Status |
|---|---|---|
| **UI Design** | Consistency, accessibility, responsiveness | Pass |
| **Functional Requirements** | All core features implemented | Pass |
| **Performance** | Meets response time targets | Pass |
| **Security** | Encryption, RBAC, secure APIs | Pass |
| **Localization** | English/Arabic fully supported | Pass |
| **Backup & Recovery** | Protocols tested and verified | Pass |
| **Scalability** | Supports load testing thresholds | Pass |

## 10. Requirements Traceability Matrix

The Requirements Traceability Matrix (RTM) ensures complete alignment between the **Software Requirements Specification (SRS)** and the **Software Design Specification (SDS)**.
It verifies that every requirement defined in the SRS is fully addressed by a corresponding design element, subsystem, component, interface, or architectural decision.

The RTM provides:

- Forward traceability (SRS → SDS → Implementation)
- Backward traceability (Design → Requirements)
- Full coverage of functional (FR) and non-functional requirements (NFR)
- Assurance that all requirements have been accounted for in the detailed design

## 10.1 Traceability Matrix

### Requirements Traceability Matrix

| Requirement ID | Requirement Description (SRS) | SDS Section(s) Addressing Requirement | Design Artifacts |
|---|---|---|---|
| FR-1 | User authentication and secure login | 7.1.1, 8.7.1, 5.3.4 | Sequence Diagram – Login, Auth Component Design |
| FR-2 | Role-based access control (RBAC) | 7.1.2, Table 13 | Roles & Permissions Matrix |
| FR-3 | Students shall view weekly class schedule | 4.3.3.1, 7.2.2, 8.7.2 | Weekly Schedule UI, Sequence Diagram – Schedule Management |
| FR-4 | Students can add reminders for classes or tasks | 4.3.4, 7.2.3, 8.7.2 | Reminder Screens, Reminder Logic Design |
| FR-5 | System sends notifications for upcoming deadlines | 7.1.3, 5.3.4 | Notification Engine, Messaging Sequence Diagram |
| FR-6 | Students shall browse available internships | 4.3.5.1, 7.2.4 | Internship Listing Screens |
| FR-7 | Students shall apply for internships and upload documents | 4.3.5.3, 7.2.4, 8.7.3 | Internship Process Sequence Diagram |
| FR-8 | Students shall track internship application status | 4.3.5.4, 7.2.4 | Internship Status Component |
| FR-9 | System shall display university announcements | 4.3.6, 7.2.5, 8.7.4 | Announcement List Screens |
| FR-10 | Users can filter announcements by category | 4.3.6.3, Table 18 | Announcements Filter UI |
| FR-11 | Users can send and receive messages from departments | 4.3.7, 7.2.6, 8.7.5 | Messaging & Notifications Sequence Diagram |
| FR-12 | Users can update profile information | 4.3.8, 7.3.1, 8.7.6 | Profile Component, Settings Component |
| FR-13 | Users can change theme and language | 4.3.9, 7.3.1 | Settings & Localization System |
| FR-14 | Users can manage notification settings | 4.3.9.3, 7.3.1 | Notification Preferences |
| FR-15 | System supports bilingual UI (Arabic/English) | 4.3.13, 9.4 | Localization Specs, RTL/LTR Support |
| FR-16 | Users can reset forgotten passwords | 4.3.1.4, 7.1.1 | Forgot Password Flow |
| FR-17 | System shall store schedule, reminders, messages, internships | 6.1, 6.2, 6.3 | Data Dictionary, ERD, Relational Schema |

| | | | |
|---|---|---|---|
| FR-18 | System shall validate user inputs and prevent errors | 8.5, 9.3 | Validation Logic, Error Handling Design |
| NFR-1 | System performance must support ≤ 300 ms response time | 9.1 | Performance Metrics |
| NFR-2 | System availability ≥ 99.5% | 9.1, Table 20 | Availability SLA |
| NFR-3 | System security must follow encryption and RBAC | 9.2, 7.1.1, 7.1.2 | Security Architecture |
| NFR-4 | System must provide reliable backup and recovery | 9.3, Table 21 | Backup & Recovery Protocol |
| NFR-5 | System must be maintainable and extendable | 9.5 | Maintainability & Extensibility Design |
| NFR-6 | UI must follow accessibility & responsive design standards | 4.2 | UI Guidelines |
| NFR-7 | Database must ensure ACID compliance | 5.3.3, 6.3 | Data Layer Design |
| NFR-8 | System should scale to 500+ concurrent users | 9.1 | Scalability Architecture |